

ArkObfuscation - a multi-layer reinforcement system designed for HarmonyOS terminal applications

Tianqi Li^{a,1}, Hanyu Zhang^{a,1}, Wenbo Chen^a, Liang Zhang^a, Lei Xue^{a,*}

^aSun Yat-sen University, Shenzhen, China

Abstract

Context: With the rapid expansion of the HarmonyOS ecosystem, the proliferation of distributed devices and ArkTS-based applications has introduced significant security risks, including reverse engineering, sensitive data leakage, and permission abuse.

Objectives: To address these challenges, we propose ArkObfuscation, a multi-layer reinforcement system designed for HarmonyOS terminal application, aiming to enhance the security and resilience of ArkTS applications against both static and dynamic attacks.

Methods: ArkObfuscation integrates multiple obfuscation and encryption techniques at the ArkTS source-code level. Specifically, the system incorporates four key mechanisms: sensitive data encryption based on the SM2 cryptographic algorithm, Bogus Control Flow, Control Flow Flattening, and Layout Confusion. These techniques collectively increase code complexity and obscure program semantics while remaining compatible with the existing HarmonyOS development workflow and not requiring to change the toolchain.

Results: Experimental evaluation demonstrates that ArkObfuscation significantly increases code complexity and effectively mitigates reverse engineering and program analysis attempts, thereby strengthening the protection of ArkTS applications.

Conclusion: The proposed system provides a practical and low-cost security reinforcement framework for HarmonyOS developers, improving application resilience while maintaining development compatibility and deployment efficiency.

1. Introduction

With the rapid proliferation of mobile and IoT ecosystems, smart terminal applications increasingly process sensitive user data and critical system functionalities. However, the openness of application distribution channels also exposes these applications to various security threats, including reverse engineering, sensitive data leakage, and permission abuse. Attackers can analyze application binaries to extract sensitive information, discover vulnerabilities, or replicate proprietary logic through advanced reverse-engineering and deobfuscation techniques [34], posing significant risks to both application providers and end users.

Code obfuscation is widely adopted as an effective defensive technique to increase the difficulty of reverse engineering while preserving program functionality. Traditional obfuscation approaches transform

*Corresponding author

Email addresses: litq25@mail2.sysu.edu.cn (Tianqi Li), hanyuzhang212@gmail.com (Hanyu Zhang), chenwb58@mail2.sysu.edu.cn (Wenbo Chen), zhangliang27@mail.sysu.edu.cn (Liang Zhang), xuelei3@mail.sysu.edu.cn (Lei Xue)

¹These authors contributed equally to this work.

program structures, control flows, or data representations to hinder static and dynamic analysis [10, 8]. In practice, many mainstream protection tools mainly rely on lightweight identifier renaming and simple code transformations, as exemplified by widely used solutions such as ProGuard [19]. Although these techniques provide basic protection, they often fail to defend against modern reverse engineering frameworks and automated analysis tools. Recent studies have pointed out that many existing obfuscators lack resilience because they deploy protections in a rigid, hard-coded manner, making the obfuscated code patterns easily learnable and distinguishable by advanced attackers [35]. This lack of flexibility and stealth underscores the need for more specialized, context-aware protection mechanisms that are deeply integrated with the application's underlying programming paradigm.

To address these limitations, we propose ArkObfuscator, a source-code-level multi-layer reinforcement system designed for HarmonyOS terminal applications. The proposed system integrates multiple protection mechanisms, including sensitive data encryption based on the SM2 elliptic curve cryptographic algorithm, control-flow obfuscation via Bogus Control Flow [11], and Control Flow Flattening [21]. In addition, Layout Confusion is introduced to obfuscate UI-related structures, further increasing reverse-engineering complexity. By combining cryptographic protection with diversified program transformations, ArkObfuscator significantly enhances the resistance of applications against automated reverse engineering and code analysis.

However, designing effective source-level protection for HarmonyOS applications presents several challenges. First, the ArkTS programming language introduces strict typing and UI-oriented programming paradigms, requiring obfuscation transformations to preserve syntactic correctness and framework compatibility. Second, naive code transformations may easily break application functionality or degrade maintainability. Third, balancing protection strength and performance overhead remains critical for real-world deployment in resource-constrained smart terminal environments.

To address these challenges, ArkObfuscator performs security reinforcement at the source-code level through AST-based program transformations. Since ArkTS is a strict superset of TypeScript, the system leverages the TypeScript compiler API to parse ArkTS source files and construct the AST representation. The resulting AST enables automated transformations that integrate multiple complementary protection techniques while supporting configurable strategies to balance security and usability.

The main contributions of this paper are summarized as follows:

- We propose ArkObfuscator, a source-code-level multi-layer security reinforcement framework for HarmonyOS terminal applications that integrates cryptographic protection with diversified program obfuscation techniques.
- We design a set of fine-grained TypeScript compiler-based program transformations, including data obfuscation, control flow obfuscation, control flow flattening and layout confusion, to significantly increase reverse-engineering difficulty while maintaining program correctness.
- We implement a prototype system and conduct comprehensive experiments on real-world HarmonyOS applications. Experimental results demonstrate that ArkObfuscator effectively increases reverse-engineering resistance while maintaining acceptable performance overhead.

2. Preliminary Knowledge

2.1. ArkTS

ArkTS is a TypeScript-based programming language designed for developing native applications on the HarmonyOS platform. As a strict superset of TypeScript, it preserves most TypeScript syntax and

programming paradigms while introducing a programming and execution model tailored to the HarmonyOS runtime environment.

Compared with general-purpose JavaScript and TypeScript, ArkTS restricts certain highly dynamic language features and emphasizes static type information, enabling more effective static analysis and compile-time optimizations. These design choices facilitate code trimming and optimization during compilation, which helps reduce runtime overhead and resource consumption on mobile and IoT devices.

ArkTS also integrates a declarative UI framework and a reactive state management mechanism, providing native support for multi-device collaborative application development within the HarmonyOS ecosystem. From a tooling perspective, ArkTS programs can be analyzed and transformed using static analysis frameworks such as ArkAnalyzer [6], which exposes intermediate representations suitable for program inspection and code-level transformations.

In this work, ArkTS serves as the target programming language for application reinforcement. All protection mechanisms proposed in this paper are applied at the ArkTS source-code level, without involving bytecode or virtual machine-level transformations.

2.2. Bogus Control Flow

Bogus Control Flow, as a code protection technique, is based on the principle of masking the actual execution path of a program by inserting unrelated control flow segments and jump logic. In traditional code, the control flow structure is usually clear and linear, while Bogus Control Flow constructs complex jump logic by introducing additional conditional statements, loops, and state variables. Although these inserted control flow segments do not affect the actual output of the program, they significantly increase the difficulty of code analysis [11].

From a technical perspective, this project's Bogus Control Flow primarily includes four methods: Meaningless Jump Obfuscation, Indirect Call Obfuscation, Finite State Machine Obfuscation, and Unconditional Jump Obfuscation. These techniques will be elaborated in §3.5. Together, they constitute a multi-layered protection system, effectively increasing the difficulty of code reverse engineering.

2.3. Control Flow Flattening

Control Flow Flattening is a powerful code obfuscation technique that restructures a program's control flow structure, transforming the original branching and looping logic into a single, flattened control flow. In traditional programs, **control flow graphs (CFG)**² typically present a tree-like or hierarchical structure, which is easy for static analysis tools to parse. However, after Control Flow Flattening, the program's CFG is simplified to a form primarily composed of a single loop body. Each basic block within this loop body uses state variables or labels to determine the jump to the next basic block, thus completely eliminating the branching hierarchy and structural features of the original code.

This flattened structure forces attackers to trace the jump relationships of each basic block one by one during reverse engineering, significantly increasing the complexity and time cost of understanding the code logic. The Control Flow Flattening implementation in this experiment mainly references a **dispatcher-based** flattened framework proposed by László and Kiss [21]. This method first decomposes the program into multiple basic blocks and manages the execution order of these blocks through a central dispatcher. The dispatcher relies on the values of state variables to determine which basic block to execute next.

²A control flow graph (CFG) in computer science is a representation, using graph notation, of all paths that might be traversed through a program during its execution.

The update logic of these state variables can be further obfuscated, for example, through encryption, dynamic calculation, or the introduction of pseudo-random factors, making it difficult for attackers to predict the jump path.

In summary, the advantage of Control Flow Flattening lies in its ability to effectively hide the program's true logic, making it difficult for static analysis tools to reconstruct the original CFG. It also presents challenges for dynamic analysis because the jump relationships are no longer intuitive. Although modern decompilation techniques attempt to recover high-level control structures from low-level control-flow graphs [39], the irregular control-flow patterns introduced by flattening still significantly increase the difficulty of structure recovery. Furthermore, this technique can be combined with other obfuscation methods (such as variable renaming, expression substitution, or constant obfuscation) to further enhance code protection [10, 8].

3. Methodology

This section first describes the overview of our obfuscation and hardening system ArkObfusc, followed by the detailed designs and implementations of its components.

3.1. Overview of ArkObfusc

The overview of our proposed obfuscation and hardening system is illustrated in Figure 1, and it consists of four major layers, including Inputs & Interface, Preprocessing & Analysis, Obfuscation Engine, and Reporting & Verification.

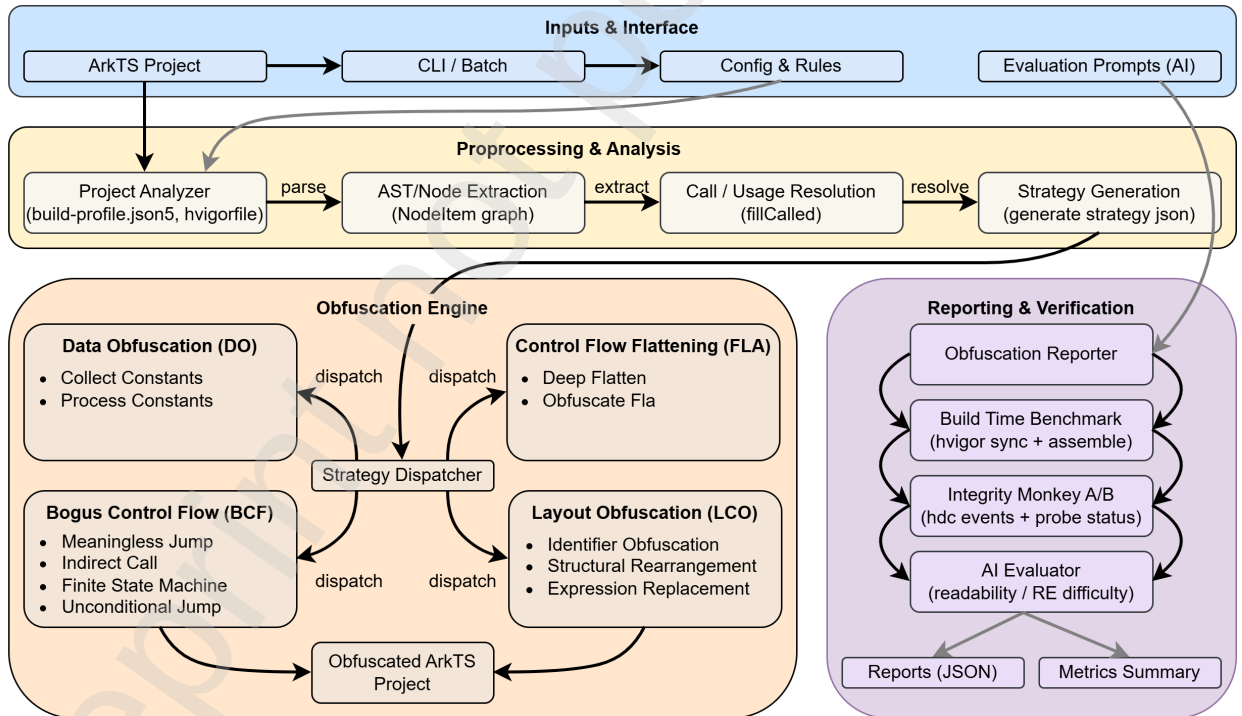


Figure 1: Architecture of ArkObfusc

Inputs & Interface: This layer serves as the system entry point. An ArkTS project is provided through a CLI or batch interface, together with configuration files and rule definitions. These configurations specify which obfuscation techniques are enabled, parameter settings, and exclusion policies. In addition, evaluation

prompts for the AI-based assessment module are defined at this stage, enabling subsequent automated quality and reverse-engineering difficulty evaluation.

Preprocessing & Analysis: This layer performs structural parsing and semantic analysis of the input project. The Project Analyzer first parses configuration files (e.g., `build-profile.json5`, `hvmgorgfile`) to understand project structure and compilation settings. Then, AST/Node extraction constructs a node-level representation (NodeItem graph) for all relevant `.ets` files. Such TypeScript compiler-based program analysis and transformation techniques are widely used in software engineering for automated code manipulation and structural analysis [12, 4, 36, 3, 23].

Based on this representation, the Call/Usage Resolution phase analyzes symbol references and invocation relationships (e.g., `fillCalled`) to build inter-procedural dependency information. These analysis results provide the foundation for safe and precise transformation. More details are in §3.2.

Strategy Generation: Instead of applying uniform obfuscation to all files, this phase generates a fine-grained obfuscation strategy for each source file and outputs a structured `strategy.json`. Based on AST features, call relationships, file paths, naming patterns, and type characteristics, the system determines which transformations are safe and suitable. For example, files containing complex asynchronous logic or specific decorator patterns may selectively disable certain transformations to prevent type conflicts or runtime errors. This context-aware strategy allocation mechanism significantly improves robustness and compatibility while maintaining strong protection. More details are in §3.3.

Obfuscation Engine: Guided by the generated strategy, the Strategy Dispatcher coordinates multiple transformation modules and applies them to the target files. The engine integrates several obfuscation techniques, including BCF (Bogus Control Flow), FLA (Control Flow Flattening), DO (Data Obfuscation), and LCO (Layout Confusion). Each module operates under strict semantic-preservation constraints and may introduce auxiliary encoding logic or helper files when necessary. The transformed files are written back to produce an obfuscated ArkTS project. More details are in §3.4–§3.7.

Reporting & Verification: After transformation, the system evaluates correctness, performance impact, and security enhancement. The Obfuscation Reporter records applied strategies and transformation statistics. Build-time benchmarking (e.g., `hvmgorg sync + assemble`) measures compilation overhead. Integrity verification (e.g., event probing and status comparison) ensures functional consistency between original and obfuscated versions. Finally, an AI-based evaluator assesses readability degradation and reverse-engineering difficulty, generating structured reports and metric summaries in JSON format. More details are in §4.

3.2. Preprocessing

In the preprocessing stage, this system performs structured parsing and dependency modeling on the original ArkTS project through two main steps, providing accurate semantic context for subsequent stages—especially identifier renaming and control flow obfuscation.

In the first phase, the system performs project analysis and constructs a hierarchical file tree. The system first recursively traverses the project root directory, parses configuration files, and extracts the source code paths of all modules. For each extracted source code path, the system performs a depth-first traversal, explicitly excluding build artifact directories and third-party dependency folders. During the traversal, the system instantiates a `ProjectNode` object for each file or subdirectory to record key information such as its path, type, and parent-child relationships—as detailed in Table 1. This step generates a standardized, hierarchical **project node tree** that accurately reflects the project’s source code layout, laying the foundation for subsequent analysis.

In the second phase, the system performs node processing. First, it filters all nodes of type `.ets` files from the constructed project node tree, converts them into an array structure, and analyzes the call relationships between nodes to generate an intermediate representation containing the complete project structure and

Table 1: Structure of the ProjectNode Intermediate Representation

Field	Description
projectPath	Absolute path to the project root directory.
type	Node type: "dir", "nets_file", or "ets_file".
node	Canonical identifier derived as <code>name.split(".")[0]</code> (e.g., Foo from Foo.ets).
name	Full file or directory name (e.g., Main.ets).
parent	Name of the immediate parent directory.
children	List of child node names (populated only for directories).
sourcePath	Absolute filesystem path of the node.
pathFromContentRoot	Relative path from the project root.
call	Array of external identifiers referenced by this node (parsed from source code).
called	Array of identifiers that reference this node (populated during dependency analysis).
oStrategy	Assigned obfuscation strategies .

dependencies. This phase ensures the system can accurately identify dependencies in the code by constructing a call graph, supporting the generation of subsequent obfuscation strategies. Simultaneously, this phase also performs static analysis of the code structure to identify key business logic and sensitive data.

In summary, the preprocessing stage outputs a structured collection of .ets files, each enriched with forward (call) and backward (called) dependency information. This call graph not only captures static module-level references but also enables context-aware obfuscation policies—such as exempting exported interfaces from renaming and ensuring cross-file symbol consistency—thereby serving as a fundamental guarantee for the correctness and effectiveness of the entire obfuscation pipeline.

3.3. Strategy Generation and Dispatch

After obtaining the complete list of project file nodes, the system enters the obfuscation strategy generation phase. The goal of this phase is to dynamically configure a set of obfuscation strategies for each .ets source file, achieving a balance between security and functionality. The system supports four basic obfuscation techniques: Control Flow Flattening (FLA), Bogus Control Flow (BCF), Data obfuscation (DO), and Layout Confusion (LCO).

By default, all obfuscation strategies except Layout Confusion are enabled. Subsequently, the strategy generation process fine-tunes the strategy set based on three dimensions of rules: semantic role recognition, path and naming pattern matching, and special directory handling.

3.3.1. Semantic Role Recognition

- For files located in the constants directory and containing keywords such as Constants in their names, only the Data obfuscation strategy is retained, while Control Flow Flattening, Bogus Control

Flow, and Layout Confusion are disabled to prevent accidental modification or invalidation of constant values.

- For files containing ArkTS@Entry decorators and Build() methods, only Layout Confusion is enabled, while all other strategies are disabled to prevent UI build logic from crashing due to changes in control flow or call structure.
- For entry classes (such as EntryAbility), build configuration classes (e.g., BuildProfile), or logging utility classes (e.g., Logger) that inherit from UIAbility, all obfuscation strategies are completely disabled to ensure stability of the application lifecycle and core infrastructure.
- If a file is a module export index (such as index.ets containing only export statements), all obfuscation strategies are disabled to maintain clarity of the module interface.

3.3.2. Path and Naming Pattern Matching

The system maintains two sets of keyword lists to identify potentially high-risk files.

- **Keywords to disable Control Flow Flattening:** service, builder, util, manager, view, etc. These files often contain complex types, asynchronous logic, or UI component properties. Applying control-flow flattening may lead to type inference failures or broken Promise chains.
- **Keywords to disable Bogus Control Flow:** BreakpointSystem, SongItemBuilder, PreferencesUtil, MediaService, etc. These files typically involve generic methods, complex return types (e.g., Promise<SongItem>), or void functions with implicit returns. Indirect-call obfuscation may introduce “method not found” errors or type mismatches.

3.3.3. Special Directory Handling

For component files located in the /view/ or \view\ directory, in addition to disabling Control Flow Flattening, Layout Confusion is also disabled to prevent mismatches between template attributes and logic-layer attributes, which could otherwise lead to abnormal UI rendering.

Ultimately, each .ets file receives a four-tuple policy configuration [bcf, do, fla, lco], explicitly indicating whether each obfuscation technique is enabled. This configuration result is serialized into a JSON file and persistently stored for subsequent obfuscation engine execution. This policy generation mechanism achieves semantic- and context-aware adaptive obfuscation, effectively avoiding compatibility issues commonly observed in traditional general-purpose obfuscators within the ArkTS ecosystem.

3.3.4. Strategy Dispatch

As shown in Figure 1, strategy dispatch is required during the transition from the Preprocessing & Analysis layer to the Obfuscation Engine layer. In this stage, ArkObfusc first loads the strategy.json file generated in the previous stage. Then, the Strategy Dispatcher (startObfusc) determines which transformation modules should be activated for each file. Based on the assigned strategy, the dispatcher invokes the corresponding obfuscation modules and applies the transformations to the parsed AST representation.

The engine currently integrates four core techniques: Data obfuscation (§3.4), Bogus Control Flow (§3.5), Control Flow Flattening (§3.6), and Layout Confusion (§3.7). Each module focuses on a different structural dimension of the program, including data representation, control-flow structure, and layout-related identifiers. The transformations are applied under strict semantic-preservation constraints to ensure that the resulting program remains functionally equivalent to the original version.

After all selected transformations are completed, the modified AST is written back to the source files, producing the obfuscated ArkTS project.

During the entire process, the ObfuscationReporter records the activated strategies, affected files, and transformation statistics. These logs are later used by the Reporting & Verification layer to generate structured reports and evaluation metrics.

3.4. Data obfuscation

For files with Data obfuscation enabled, this obfuscation tool implements two obfuscation mechanisms. One is constant obfuscation for numeric and string constants, and the other is function name obfuscation for function names.

3.4.1. Constant Obfuscation

For numeric and string constants, this obfuscation tool employs a two-stage data obfuscation mechanism to prevent them from appearing in plaintext in the source code or decompiled output. As shown in Algorithm 1, this mechanism is based on the lightweight SM2 elliptic curve cryptography algorithm, statically encrypting strings and numeric constants and dynamically decrypting them at runtime.

- **Step 1: Constant Collection and Encryption.**

In Step 1, the project first scans all `Constants.ets` files to identify statically defined constants (e.g., URLs, API keys, UI configuration values) and skips those in the whitelist. Then, the project applies different encryption methods based on the constant type: for numeric constants, the expression is first evaluated and then encrypted using SM2; for string constants, they are split into two bytes according to the Unicode encoding of the characters and encrypted byte-by-byte into `EncryptedChar` objects. The encryption results are encoded using structured types: numeric constants use the `EncryptedNumber` type; string constants use the `EncryptedChar []` type, which is an array of `EncryptedChar` objects.

Next, a unique key pair is generated for each constant, and the encrypted value and private key are stored in the global `globalConstants` object. Finally, the file is modified to replace the constant definitions with the encrypted types and values.

- **Step 2: Constant Dot Replacement**

In Step 2, the system excludes the constant definition file `Constants.ets`, iterates through all `.ets` files that reference constants, and replaces constant accesses with corresponding decryption function calls. Replacement supports multiple access methods: direct access (`ClassName.CONSTANT`), alias import access (`Alias.CONSTANT`), and array access. The replacement result is a function call in the form of `decryptStringConstant(privKey, encryptedData)` or `decryptNumberConstant(privKey, encryptedData)`, where `privKey` is the private key generated in Step 1, and `encryptedData` is the encrypted constant value.

To ensure correctness, the system employs a context-aware filtering mechanism: skipping enumeration attribute assignments to avoid erroneous replacements; and also skipping constant references in comments and string literals.

After replacement, the system automatically adds the corresponding decryption function import statement to the top of the file and ensures that the decryption tool file `sm2.ets` has been copied to the target directory. In addition, the system not only processes the current file, but also recursively processes the called files, ensuring that constant references in the dependency relationships are correctly replaced.

Algorithm 1 Data Obfuscation: Two-Phase Constant Encryption and Replacement

Require: Project node set $\mathcal{N}$, whitelist $\mathcal{W}$, pattern set $\mathcal{E}$

Ensure: Obfuscated source files with constants replaced by runtime decryption calls

```
1: Initialize  $\mathcal{G} \leftarrow \emptyset$ 
2:                                     ▶ Stage 1: Constant Collection
3: for each node  $n \in \mathcal{N}$  with  $\text{basename}(n.\text{path})$  ending “Constants.ets” do
4:    $\text{content} \leftarrow \text{ReadFile}(n.\text{path}); \text{className} \leftarrow \text{ExtractClassName}(\text{content})$ 
5:   if  $\text{className} = \emptyset$  then
6:     continue
7:   end if
8:   for each  $\langle \text{name}, \text{type}, \text{value} \rangle$  in  $\text{content}$  do
9:     if  $\text{name} \in \mathcal{W}$  or matches  $\mathcal{E}$  then
10:      continue
11:    end if
12:     $(d, P) \leftarrow \text{GenKeyPair}(); C \leftarrow \text{Encrypt}(P, \text{value}, \text{type})$ 
13:     $(d, P) \leftarrow \text{GenKeyPair}()$ 
14:    if  $\text{type} = \text{“number”}$  or  $\text{value}$  is numeric then
15:       $C \leftarrow \text{EncryptNumber}(P, \text{Eval}(\text{value})); \tau \leftarrow \text{“number”}$ 
16:    else
17:       $C \leftarrow \text{EncryptString}(P, \text{value}); \tau \leftarrow \text{“string”}$ 
18:    end if
19:     $\mathcal{G}[\text{className.name}] \leftarrow (C, \tau, d)$ ; replace declaration in  $\text{content}$ 
20:  end for
21:   $\text{WriteFile}(n.\text{path}, \text{content})$ 
22: end for
23:  $\text{SerializeMetadata}(\mathcal{G})$ 
24:                                     ▶ Stage 2: Usage Replacement
25: for each node  $n \in \mathcal{N}$  s.t.  $\text{basename}(n.\text{path}) \neq \text{“Constants.ets”}$  do
26:    $\text{content} \leftarrow \text{ReadFile}(n.\text{path}); \alpha \leftarrow \text{BuildAliasMap}(\text{content}); \text{modified} \leftarrow \text{False}$ 
27:   for each  $\kappa \in \mathcal{G}$  with  $(C, \tau, d)$  do
28:      $\text{identifiers} \leftarrow \{\text{Split}(\kappa).\text{className}\} \cup \{\alpha[\text{className}]\}$ 
29:     Replace  $\text{id.constName}$  and bare  $\text{constName}$  (if destructured) with  $\text{Decrypt}(d, C)$ , skip enum
    contexts
30:     if replacement occurred then
31:        $\text{modified} \leftarrow \text{True}$ 
32:     end if
33:   end for
34:   if  $\text{modified}$  then
35:      $\text{PrependImport}(\text{content}); \text{WriteFile}(n.\text{path}, \text{content})$ 
36:   end if
37: end for return Obfuscated project
```

3.4.2. Function Name Obfuscation

Function Name Obfuscation replaces function names with randomly generated identifiers and maintains a mapping between the original names and the obfuscated names. This method effectively prevents the leakage of function call relationships.

3.5. Bogus Control Flow

For files with Bogus Control Flow enabled, this project employs a layered, progressive strategy. In three main phases, the project uses various techniques to gradually increase the code obfuscation strength, making reverse engineering difficult.

In the first phase, the project primarily implements basic control flow obfuscation at the method level. First, **Meaningless Jump Obfuscation** are used to randomly insert 3-5 spurious conditional branches containing complex operations into the method. These do not affect the original logic but increase the difficulty of static analysis. Then, the project replaces direct calls with indirect calls via proxy methods through **Indirect Call Obfuscation**, increasing the complexity of the call chain. Finally, the project uses **Finite State Machine Obfuscation** to implant pseudo-state machine logic into the target function, making control flow analysis even more difficult. These three methods will be explained in detail in §3.5.1, §3.5.2, and §3.5.3, respectively.

In the second phase, the project performs deep control flow flattening on files that enable both Bogus Control Flow and Control Flow Flattening. Specifically, the **Control Flow Flattening** method first transforms sequential execution into a dispatcher-controlled jump structure, then adds decryption logic to the dispatcher expression to further hide the dispatcher's jump logic. This method will be explained in detail in §3.6.

In the third phase, the project performs structural obfuscation on the files. Specifically, the project introduces a random number of jump steps using **Unconditional Jump Obfuscation** to increase structural complexity and make the control flow graph more complex. This method will be explained in detail in §3.5.4.

Through the methods described above, the Bogus Control Flow approach constitutes a complete security protection chain. This protection system not only effectively resists static analysis but also, to a certain extent, resists dynamic debugging and runtime analysis, ultimately achieving multi-dimensional defense for code protection, making it difficult for attackers to breach the defenses using a single method. Simultaneously, this technology increases the cost of reverse engineering the code by enhancing its complexity and unpredictability, providing an effective technical means for protecting software intellectual property rights. A schematic diagram of the four methods used in Bogus Control Flow is shown in Figure 2.

3.5.1. Meaningless Jump Obfuscation

This obfuscation strategy inserts a random number of jump steps into non-empty methods, combining bitwise and string operations to construct a complex control flow path.

While the obfuscation logic involves complex arithmetic and control flow operations, it is semantically preservative because all modifications are confined to local variables, and these changes do not affect the execution order or result of the original business logic. Furthermore, the original method body is appended after all step blocks to ensure complete execution.

This mechanism increases the difficulty for decompilers and static analysis tools to identify the true business logic, effectively increasing the difficulty of reverse engineering.

3.5.2. Indirect Call Obfuscation

The Indirect Call Obfuscation strategy introduces a proxy layer and runtime verification mechanism to convert original method calls into indirect call patterns. This makes it impossible for static analysis tools to

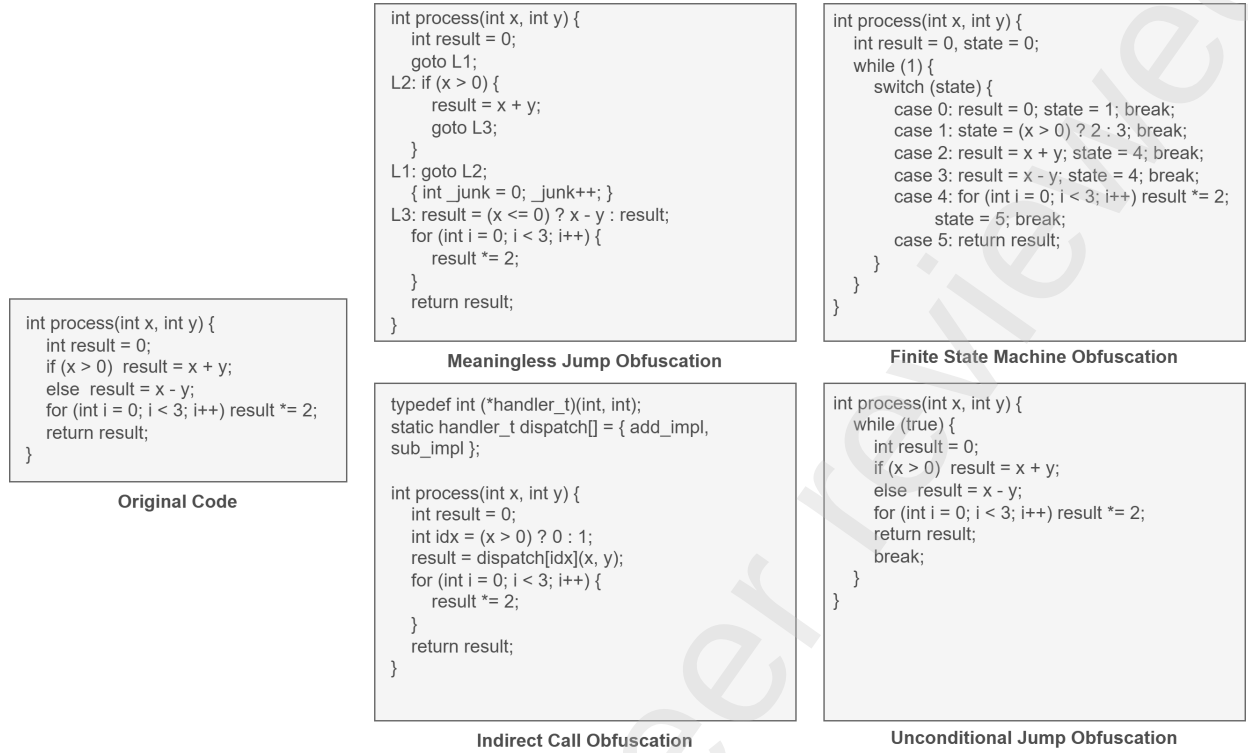


Figure 2: Comparison of four Bogus Control Flow obfuscation strategies applied to the same function

directly identify method call relationships, significantly increasing the difficulty of reverse engineering and ensuring that static analysis tools cannot directly identify the original method calls.

The system first dynamically assigns a unique CheckCode to each method in the class using the following formula:

$$\text{CheckCode} = B + \text{MethodIndex}$$

where MethodIndex represents the method declaration order (starting from 0), and B is a configurable base offset chosen to ensure uniqueness and avoid collision with reserved or commonly used values in the target program. Then, a proxy method is injected into the target class. This proxy method accepts CheckCode as its first parameter and preserves the return type signature of the original method. Subsequently, the system iterates through all method call points in the target file and replaces direct calls with indirect calls containing the check code.

At runtime, the proxy method performs verification: if the provided CheckCode matches the expected value, the call is forwarded to the original method; otherwise, a type-safe default value (e.g., "" for string, [] for arrays, false for boolean) is returned. This ensures semantic correctness and avoids runtime crashes while preserving obfuscation guarantees.

3.5.3. Finite State Machine Obfuscation

The core objective of Finite State Machine Obfuscation is to significantly increase the complexity of the control flow graph by embedding **pseudo state machine** (PSM) logic into target functions, thereby interfering with the parsing capabilities of the TypeScript AST parser while ensuring the semantic integrity of the original functionality.

First, the system parses the target source file using the TypeScript AST parser, generating an **Abstract Syntax Tree** (AST) to identify key structural nodes such as function and method declarations. This AST-

based parsing method ensures the accuracy of code transformation, enabling the location of obfuscation logic insertion without compromising the original semantics. After parsing, the system checks if the target file contains necessary utility class references; if missing, it automatically inserts the corresponding import statements to ensure that subsequent obfuscation logic relies on a unified utility class implementation.

During the obfuscation logic insertion phase, Finite State Machine Obfuscation employs a utility class-based PSM generation strategy. For each function or method, the system generates a unique flow identifier to identify a specific PSM instance, combining it with timestamps and version information to enhance the dynamics of obfuscation. The PSM logic consists of multiple state transition checks and transition operations, implemented through the state analyzer and flow analyzer in the utility class. Each PSM contains several state transition check steps, each including condition checks and error handling logic. However, this logic has no actual dependency on the original business code; it is only used to construct complex control flow paths. By inserting this PSM code at the beginning of function bodies, the system can significantly expand the number of nodes and edges in the control flow graph, creating numerous invalid paths and increasing the difficulty for decompilers to reconstruct the true logic.

In the final stage of code transformation, the system uses an AST converter to regenerate the modified code and writes the result back to the source file. This process ensures seamless embedding of obfuscation logic while preserving the functional integrity of the original code.

3.5.4. Unconditional Jump Obfuscation

Unconditional Jump Obfuscation is a control flow obfuscation technique that introduces an artificial loop structure around the method body to disrupt static analysis of program behavior. The core idea is to wrap the original method logic in a loop with a condition that appears to be always true, and to place an unconditional exit statement immediately after the original logic. At runtime, execution enters the loop, runs the original statements once, and then exits via the exit statement; the loop body is thus executed exactly once, preserving the original semantics.

From the perspective of static analysis, however, the control flow graph (CFG) is altered. The analyzer encounters a loop whose condition is syntactically constant, which typically implies an unbounded iteration. This creates a mismatch between the apparent structure (an infinite loop) and the actual execution (single pass with immediate exit). Tools that rely on structural analysis or symbolic execution must account for the loop back edge and the unconditional exit, increasing the complexity of path analysis and potentially leading to conservative or imprecise results [29]. Furthermore, the introduction of an extra loop node and an unconditional jump obscures the original linear or branching structure of the method, making it harder to reconstruct the intended control flow.

Compared with more complex techniques such as control flow flattening, Unconditional Jump Obfuscation achieves a balance between obfuscation strength and implementation simplicity. It requires minimal transformation of the original code, introduces no additional variables or predicates, and preserves the exact execution semantics. As a result, it serves as a lightweight complement to other control flow obfuscation strategies within the BCF framework, effectively interfering with static analysis while maintaining low overhead and high compatibility with the target language.

3.6. Control Flow Flattening

For files with Control Flow Flattening enabled, this project first analyzes and filters the methods that require control flow flattening, excluding asynchronous methods, methods with Promise return types, and methods with complex return types to ensure type safety after transformation.

Next, the project applies deep flattening to transform the original control flow into a dispatcher-driven switch-case structure. The system first identifies split points (e.g., variable declarations and return

statements) and partitions the method body into multiple basic blocks. A random label is generated for each basic block, and a label array constant is constructed to represent the execution order.

A `while` loop is then introduced as the main dispatcher, with the loop condition defined as the tag array length being greater than zero. Inside the loop, a `switch` statement selects the basic block to execute based on the encoded tag value retrieved from the array. Each case value is encrypted using a custom encoding function that generates a 64-bit hexadecimal string through combined mathematical transformations, modified Base64 encoding, and SHA-256 hashing. The generated case statements are randomly shuffled to further disrupt structural predictability. A `default` branch is included to prevent infinite loops in case of mismatches.

To further increase reverse engineering resistance, the dispatcher tag array introduced by Control Flow Flattening is also encrypted. Instead of storing raw label identifiers, each array element is pre-encoded using a lightweight symmetric transformation before insertion. During execution, the dispatcher dynamically decodes the current element prior to matching it against the encrypted case values. This dual-layer protection conceals both the control transfer logic and its underlying state representation, significantly increasing the cost of static analysis.

Finally, Control Flow Flattening post-processing is performed. By detecting the specific `while-switch` encode structure, the system applies additional XOR encryption to the tag array elements and injects corresponding decryption logic into the `switch` expression. The transformed code is written back to the source files using automated tooling, and auxiliary files are dynamically generated to support the encryption routines, ensuring seamless integration into the build process. A schematic diagram of Control Flow Flattening is shown in Figure 3.

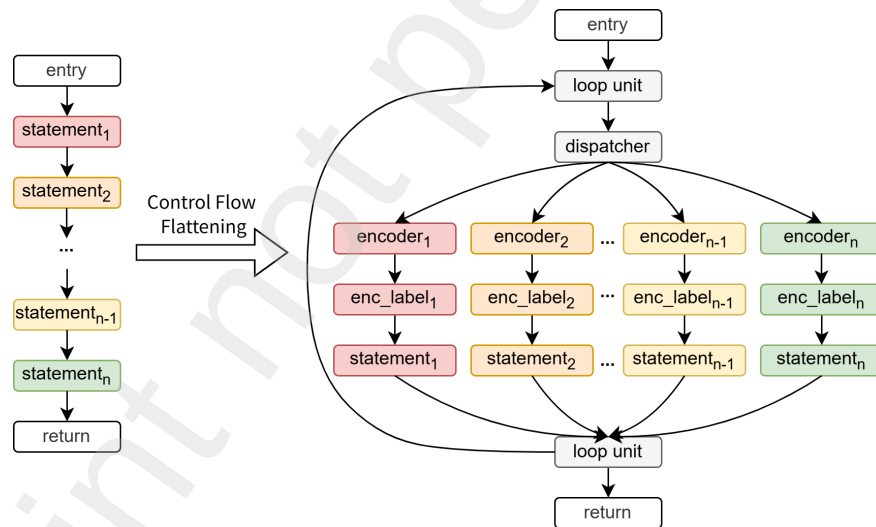


Figure 3: Control Flow Flattening strategy

3.7. Layout Confusion

For files with Layout Confusion enabled, this project implements a selective obfuscation strategy: it only obfuscates the component's internal state properties, without affecting external interface properties and properties with default values. This enhances security while avoiding property name mismatches during component calls.

Layout Confusion is typically applied to layout-related source files, such as UI component definition files (e.g., `.ets` files in ArkTS projects) that describe visual structure, state binding, and declarative rendering

logic. These files frequently contain state-driven layout expressions, two-way bindings, and reactive variables directly associated with user interaction and UI presentation. As layout files are more likely to expose structural and semantic clues about application behavior, they represent a high-value target for reverse engineering and therefore benefit significantly from controlled obfuscation.

The core of Layout Confusion lies in accurately identifying the types of properties that require obfuscation and safely replacing variable names. The system first identifies all decorator declarations using regular expressions, then categorizes them according to decorator type: internal state decorators such as `@State`, `@StorageLink`, and `@StorageProp` are marked as obfuscatable objects, while external interface decorators such as `@Link` and `@Prop`, as well as properties with default values, are excluded from obfuscation. For properties requiring obfuscation, the system generates a unique and random string identifier as a new variable name and replaces it in all relevant references throughout the code, including `this.` access and `$` two-way binding syntax. The entire process strictly adheres to the principle of semantic integrity, ensuring that the obfuscated code is logically identical to the original code, only changing the variable namespace without affecting its functional behavior.

4. Evaluation

In this section, we evaluate the performance of ArkObfuscator via answering the following four evaluation questions.

RQ1: How effectively does ArkObfuscator obfuscate ArkTS source code?

RQ2: How effectively does ArkObfuscator increase the difficulty of reverse engineering?

RQ3: How does ArkObfuscator perform compared to existing ArkTS obfuscation and hardening solutions in terms of overall obfuscation effectiveness and capability coverage?

4.1. Dataset Generation

To comprehensively evaluate the obfuscation effectiveness of ArkObfuscator under diverse application scenarios, we construct a benchmark dataset based on real-world HarmonyOS ArkTS projects. Instead of relying on manually assigned security labels or rule-based hardening recommendations, our evaluation focuses on measuring the behavioral and structural changes introduced by obfuscation across different functional categories of ArkTS applications and source files.

Collection and Categorization. We collect more than 100 open-source ArkTS demo projects from the Huawei Developer Platform (HarmonyOS NEXT Codelabs), after excluding demos implemented primarily in other programming languages. The selected projects represent the majority of practical development scenarios in the HarmonyOS ecosystem and are categorized into five high-level functional domains: *AI*, *Application Framework*, *Application Services*, *System*, and *Media*. These categories cover typical use cases ranging from user interaction and system services to multimedia processing and intelligent functionalities.

- **AI:** The AI demos cover everything from underlying CANN/AscendC operator support and model testing to multimodal AI capabilities such as vision, speech, and natural language understanding. They also include system-level AI services such as intent frameworks, comprehensively demonstrating the AI development capabilities on HarmonyOS from model deployment to scenario-based applications.
- **Application Framework:** The App Framework class demo comprehensively covers the core capabilities of HarmonyOS application development, from ArkTS syntax, ArkUI components and layouts, state

management, to UIAbility lifecycle, performance optimization, automated testing, etc., fully demonstrating the full-process framework capabilities from basic UI construction to complex application development.

- **Application Services:** The App Service demo comprehensively covers various system-level and business-level service capabilities under the HarmonyOS ecosystem, from basic services such as advertising, payment, and maps to advanced capabilities such as integrated scenarios, cloud development, distributed databases, and application markets. It also integrates development tools such as DevEco Testing, fully demonstrating how to build full-scenario application services from commercial monetization and user interaction to enterprise management by calling various kits and APIs.
- **Media:** The Media class demo is based on HarmonyOS's multimedia framework and dedicated kit, covering core media processing scenarios such as QR code recognition, video playback, and ringtone settings. It fully demonstrates how to call system media capabilities in ArkTS development to achieve rich audio, video, and interactive experiences.
- **System:** The System class demo comprehensively covers the core system capabilities of HarmonyOS, from network communication, security authentication, and system services to development tools and native interaction, fully demonstrating how to call the underlying system capabilities to build highly reliable, highly secure, and cross-scenario complex applications.

Due to the limited availability of large-scale open-source HarmonyOS terminal applications written purely in ArkTS, the total number of eligible projects is inherently constrained. Nevertheless, the selected demos are officially maintained, well-structured, and widely used as reference implementations, making them representative samples of real-world ArkTS development practices.

Evaluation Strategy. Unlike traditional datasets that assign predefined hardening strength labels, we adopt a *pairwise evaluation strategy*. For each project, we apply ArkObfuscator to generate its obfuscated counterpart and conduct comparative analysis between the original and obfuscated versions. All evaluation metrics are computed on each original–obfuscated project pair, and the results are aggregated and analyzed according to their functional categories. This design allows us to directly observe the impact of obfuscation under different application contexts, without introducing subjective labeling bias.

Function-Level Classification. To further enable fine-grained analysis, we classify ArkTS source files and their contained functions or methods into eleven functional categories according to their roles in typical HarmonyOS applications:

- **Entryability:** Application entry points and lifecycle management (e.g., `EntryAbility.ets`).
- **Pages:** Page-level UI implementations (e.g., `HomePage.ets`).
- **View:** View components responsible for UI rendering.
- **Components:** Reusable UI or functional components.
- **ViewModel:** View data models and state management logic.
- **Controller:** Business controllers coordinating core logic.
- **Utils:** Utility functions and helper modules (e.g., logging, networking).

- **Constants:** Constant definitions and configuration values.
- **Model:** Data model classes.
- **Database:** Database access and persistent storage logic.
- **BusinessLogic:** Other uncategorized business logic by default.

This classification scheme is designed to reflect the architectural patterns encouraged by ArkTS and the HarmonyOS application framework, and it covers nearly all common development scenarios encountered in practice.

Rationale and Representativeness. Although the dataset size is modest compared to large-scale benchmarks in other ecosystems, it achieves high coverage across both project-level functionalities and function-level roles. By evaluating obfuscation effects across different application domains and code responsibilities, the dataset enables a comprehensive assessment of how ArkObfuscation behaves under heterogeneous environments. This approach provides a realistic and reproducible evaluation setting for ArkTS code obfuscation, capturing both global application behavior and local structural transformations introduced by obfuscation.

4.2. Experimental Settings

4.2.1. Hardware Configuration

All experiments were conducted on a laptop equipped with an Intel Core i7-13700H processor (14 cores and 20 threads), 16 GB of RAM, running Windows 11 (64-bit).

4.2.2. Software Environment

The experiments were implemented using Node.js v20.18.0 and TypeScript v4.9.5. All measurements were performed on a single machine to eliminate cross-platform variability.

4.2.3. Evaluation Infrastructure

Static Analysis Tools. Static metrics are collected using multiple open-source analysis tools across different abstraction levels. Cyclomatic Complexity (CC) is computed using an open-source analyzer.³ Control-flow graph (CFG) structural metrics are extracted using the ArkTS static analysis framework *arkanalyzer*,⁴ including node count, edge count, branch density, average in-degree, average out-degree, number of methods, and maximum nesting depth. AST node count is obtained via the official TypeScript compiler API. Code similarity is measured using JPlag.⁵ File-level entropy is calculated using a CLI entropy tool.⁶ For bytecode-level inspection, the official ArkTS disassembler *ark_disasm* is used to convert compiled .abc binaries into readable .asm format.

LLM Configuration. We employ three large language models, namely *DeepSeek-V3.2*, *DeepSeek-V3.1*, and *Doubao-Seed-1.8*, accessed via the Volcengine API platform. All models are invoked under identical prompting settings. The prompt explicitly defines the evaluation metrics and corresponding score ranges to ensure consistent interpretation across models. To maintain comparability, identical inputs and scoring criteria are used for all LLM-based assessments.

³<https://github.com/pilotpirxie/cyclomatic-complexity>

⁴<https://gitcode.com/openharmony-sig/arkanalyzer>

⁵<https://github.com/jplag/JPlag>

⁶<https://github.com/merces/entropy>

4.2.4. Compared Obfuscation Tools

We primarily compare ArkObfuscator against ArkGuard as a reproducible baseline for ArkTS obfuscation. Commercial hardening products are discussed only at the capability level due to their closed-source nature. The detailed baseline selection and comparison rationale are provided in Section 4.5.

4.3. The effectiveness of ArkObfuscator in obfuscating ArkTS source code(RQ1)

In this section, we conduct a comprehensive evaluation of ArkObfuscator from two core perspectives: first, verifying whether the obfuscation tool preserves the semantic and functional integrity of ArkTS applications to ensure the obfuscated programs can run normally; second, quantifying the obfuscation effectiveness of ArkObfuscator at the source code dimension through multiple structural and syntactic metrics, to assess the degree of complexity enhancement and structural disruption of the source code after obfuscation.

Specifically, we first design targeted test metrics to complete the semantic and functional preservative test, and then construct a multi-dimensional quantitative evaluation system for the source code level. This evaluation design is motivated by prior studies showing that obfuscation transformations may alter program behaviors or even introduce exploitable vulnerabilities if not carefully validated [24]. Therefore, assessing semantic preservation and runtime stability is a fundamental step in evaluating the practicality of an obfuscation system.

4.3.1. Semantic and Functional Preservative Test

Whether the obfuscation tool can preserve the semantic and functional integrity of the original application is the basic criterion for evaluating its practical value. We use the official HarmonyOS development platform DevEco Studio to compare the ArkTS applications before and after obfuscation, and design three core test metrics: Compilation Pass Rate, Time Performance, and Functional Completeness to complete the preservative test, with the test results based on all applications in the dataset.

Compilation Pass Rate and Compilation Time. Whether the obfuscated application can be successfully compiled is a key criterion for evaluating the obfuscation tool’s capabilities. It directly reflects whether the tool can preserve the semantics and logic of the source code while considering the syntactic features of the obfuscated language. We conducted batch tests on the obfuscated dataset, and the results show that all obfuscated project code can be compiled.

Table 2: Comparison of compilation times for different modules (before and after obfuscation)

Category	Raw Compilation Time (s)	Obfuscated Compilation Time (s)	Change(%)
AI	17.6	18.1	2.84
App Framework	19.1	19.3	1.05
App Service	20.9	22.2	6.22
Media	30.3	31.9	5.28
System	21.1	21.7	2.84

Furthermore, the runtime of obfuscated applications may increase, and excessive overhead could lead to compilation timeouts or crashes during real-world device testing. Therefore, as shown in Table 2, we statistically analyzed the average simulator test time of the applications in the dataset before and after obfuscation to evaluate the runtime overhead introduced by the obfuscation and assess its practical impact. The results indicate that the additional overhead remains within an acceptable range for practical deployment.

Functional Completeness. Functional completeness is the most important criterion for measuring obfuscation effectiveness. We extracted samples from the five categories of the dataset and performed functional completeness tests on the obfuscated version (B) and the original version (A). Specifically, we generated a sequence of random events using test seeds and then executed the events. The event success rate represents the probability that an event was successfully generated, while the probe success rate represents the probability that an event was successfully executed. As shown in Table 3, all extracted samples passed the functional integrity test, demonstrating that ArkObfusc performs excellently in terms of semantic and functional preservation.

Table 3: Functional completeness test results (pass rate and success rates of event generation and execution for obfuscated vs. original code across five categories). Abbreviations: ESR = Event Success Rate; PSR = Probe Success Rate.

Category	Sampled Items	Test Seeds	Pass Rate	ESR	PSR
AI	5/16	50	100% (50/50)	1.0 / 1.0	1.0 / 1.0
App Framework	5/44	50	100% (50/50)	1.0 / 1.0	1.0 / 1.0
App Service	5/23	50	100% (50/50)	1.0 / 1.0	1.0 / 1.0
Media	4/4	40	100% (40/40)	1.0 / 1.0	1.0 / 1.0
System	5/12	50	100% (50/50)	1.0 / 1.0	1.0 / 1.0
Total	24/99	240	100% (240/240)	—	—

Stability Test. In this step, we designed an A/B differential test on a real device to verify whether the obfuscated version (B) and the original version (A) exhibit identical runtime behavior. Such validation is important because previous studies have shown that obfuscation transformations may unintentionally affect software reliability or change vulnerability behaviors [24].

The test results are shown in Table 4. The overall pass rate is $220/240 \approx 91.7\%$. The results remain stable under randomized monkey testing before and after obfuscation. Specifically, the AI, App Service, and Media categories achieved a 100% pass rate, while App Framework and System each had 10 failing seeds, resulting in an 80% pass rate.

Importantly, for all failing seeds, the original and obfuscated versions exhibited identical failure behaviors (e.g., crash type and triggering path). This indicates that these failures originate from the applications themselves rather than being introduced by the obfuscation process. Therefore, the observed failures cannot be attributed to ArkObfusc and do not affect its functional correctness or stability guarantees.

4.3.2. Source Code-level Obfuscation Performance Evaluation

On the basis of verifying that ArkObfusc has good semantic and functional preservation, this part further evaluates the obfuscation effectiveness of ArkObfusc at the ArkTS source code level through a set of structural and syntactic metrics commonly used in software complexity analysis. These metrics characterize the control-flow structure, syntactic scale, structural redundancy and syntactic similarity of the program. Many of them are widely used in classical software complexity and maintainability analysis, such as Halstead complexity metrics [15] and the Maintainability Index [25]. They comprehensively reflect the complexity enhancement and structural transformation effect of the source code after obfuscation from the reverse engineer’s perspective.

Table 4: Stability test results under monkey testing (pass rate and success metrics comparing obfuscated vs. original code across five categories). Abbreviations: ESR = Event Success Rate; PSR = Probe Success Rate.

Category	Sampled Items	Total Seeds	Pass Rate	ESR	PSR
AI	5/16	50	100% (50/50)	1.0/1.0	1.0/1.0
App Framework	5/44	50	80% (40/50)	0.802/0.802	0.8/0.8
App Service	5/23	50	100% (50/50)	1.0/1.0	1.0/1.0
Media	4/4	40	100% (40/40)	1.0/1.0	1.0/1.0
System	5/12	50	80% (40/50)	0.802/0.802	0.8/0.8
Total	24/99	240	91.7% (220/240)	—	—

Cyclomatic Complexity (CC). Cyclomatic Complexity is a standardized software complexity metric originally proposed by McCabe [22]. It quantifies the number of linearly independent execution paths in a program based on its control-flow graph. A higher CC value indicates a more complex decision structure and a larger number of feasible execution paths, which directly increases the cognitive burden for reverse engineers attempting to understand program logic, reason about path conditions, and reconstruct high-level semantics from low-level representations.

In the context of reverse engineering, elevated cyclomatic complexity is widely regarded as an effective indicator of increased analysis difficulty, as it complicates control-flow reconstruction and hampers both manual inspection and automated static analysis tools. Obfuscation techniques that introduce additional branches, opaque predicates, or artificial control-flow structures typically result in a significant increase in CC.

In our evaluation, CC is computed at the ArkTS source-code level using an open-source static analysis tool for TypeScript and JavaScript programs.

As shown in Table 5, obfuscation significantly increases control-flow complexity in four out of five categories.

The average complexity increases accordingly (e.g., Media 2.77→5.08, App_service 2.66→4.58), indicating structural expansion within individual functions rather than merely increased function counts. Meanwhile, maximum complexity values remain largely unchanged, suggesting that the amplification is broadly distributed rather than driven by extreme outliers.

The AI category exhibits a decrease in average CC after obfuscation. This deviation is primarily attributed to syntactic compatibility limitations of the CC extraction tool, which is designed for standard TypeScript syntax, whereas ArkTS introduces extended language constructs. Certain ArkTS-specific control-flow patterns may therefore be partially simplified or omitted during static parsing, leading to conservative CC estimates in this category.

We manually inspected representative samples from this category and confirmed that no actual control-flow simplification was introduced by the obfuscation process.

Importantly, this effect is confined to a single dataset and does not alter the overall trend observed across other categories. Moreover, maximum CC values remain stable, indicating that no extreme structural simplification occurs after obfuscation. Overall, the results demonstrate systematic control-flow complexity amplification while maintaining bounded per-function growth.

Table 5: Cyclomatic Complexity Metrics Before and After Obfuscation (Including Variations). Abbreviations: CC = Cyclomatic Complexity.

Category	State	Functions Count	Total CC	Average CC	Maximum CC
AI	Original	645	1651	2.5600	35
	Obfuscated	436	1062	2.4400	35
		-209	-589	-0.1200	0
App Framework	Original	3684	8793	2.3868	99
	Obfuscated	4242	17324	4.0839	99
		558	8531	1.6971	0
App Service	Original	2874	7654	2.6600	108
	Obfuscated	3395	15543	4.5800	116
		521	7889	1.9200	8
Media	Original	418	1157	2.7679	35
	Obfuscated	490	2489	5.0796	35
		72	1332	2.3117	0
System	Original	1214	3047	2.5099	35
	Obfuscated	1317	4881	3.7062	35
		103	1834	1.1963	0

CFG Structural Metrics. Control Flow Graph (CFG) structure metrics—such as node count, edge density, and branching complexity—provide a quantitative means to capture program size and control-flow disruptions introduced by obfuscation. Control-flow obfuscation techniques typically increase structural complexity by introducing additional basic blocks, conditional branches, and indirect control transfers [8, 1, 37]. Such graph-theoretic features have been widely adopted in security analysis, including malware detection [33] and obfuscation identification [7].

In this work, we leverage CFG-based metrics at the source level to assess how obfuscation transformations alter graph-theoretic properties of ArkTS code. Specifically, we employ ARKANALYZER [6] to extract CFG-level metrics from both original and obfuscated versions of representative ArkTS files across five distinct datasets.

As shown in Figure 4, obfuscation leads to substantial increases in CFG node count across all datasets, with growth rates ranging from 96% to 200%. The number of edges exhibits even more pronounced expansion, increasing by over 250% in all cases—reaching as high as 610% in the App Framework dataset. These results indicate that obfuscation introduces a large number of additional basic blocks and complex control-flow transformations.

Structural complexity is also significantly amplified. After obfuscation, the average number of branch nodes increases by more than 285%, with the App Framework dataset showing an increase from 1.72 to 12.175—a growth rate of 608%. Concurrently, branch density rises by 74% to 223%, reflecting a substantial

insertion of conditional branches and control-flow redirections.

Notably, while the average number of methods remains relatively stable (with changes within $\pm 20\%$), the overall CFG structure becomes markedly more intricate due to increased node and edge counts. This demonstrates that obfuscation primarily expands the control-flow graph through internal restructuring rather than method proliferation.

In summary, obfuscation induces significant structural inflation in the CFG, characterized by expanded node and edge counts, heightened branching complexity, and increased graph density, thereby enhancing resistance to static analysis.

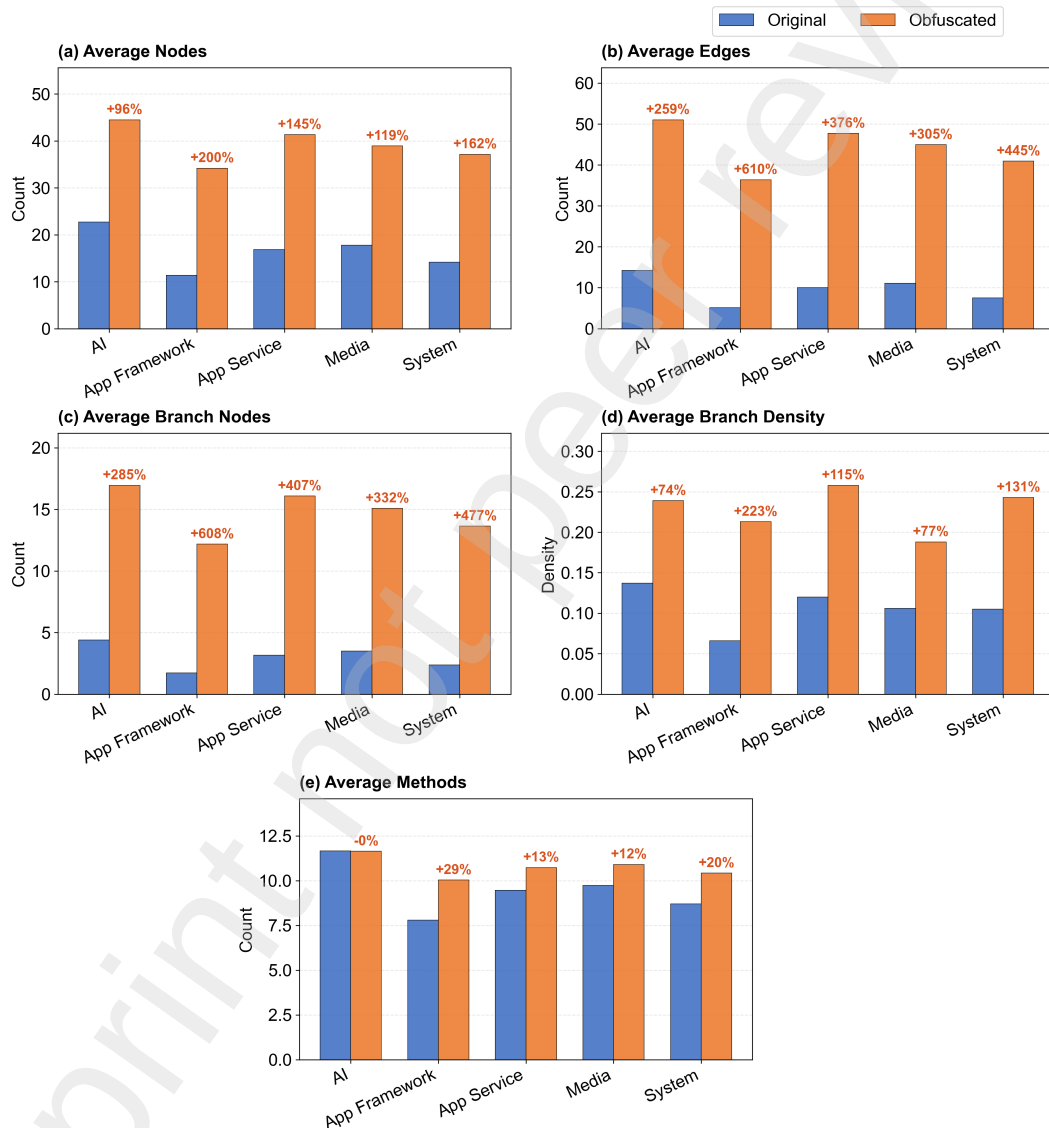


Figure 4: CFG Structural Metrics Before and After Obfuscation (Including five dimensions)

AST Node Count. AST Node Count quantifies syntactic growth at the abstract syntax tree (AST) level, capturing code inflation and structural noise introduced by obfuscation transformations. An increase in AST nodes typically indicates the insertion of auxiliary statements, expanded control constructs, and fragmented

expressions, all of which reduce code readability and increase analysis overhead.

Given that ArkTS is a strict superset of TypeScript, the source files are made compatible with the TypeScript toolchain by renaming the file extension to .ts. We then utilize the official TypeScript compiler API to parse the transformed files and compute AST-level statistics, enabling precise and fine-grained measurement of structural growth.

As shown in Figure 5, obfuscation leads to a consistent and substantial increase in AST size across all categories. On average, the number of AST nodes per file grows by roughly $2.5\times$ – $4\times$ after obfuscation.

The median growth rates closely follow the average trend, indicating that the expansion is broadly distributed rather than driven by isolated outliers. Meanwhile, the wide min–max ranges suggest the presence of heavy-tailed expansion in certain files, reflecting heterogeneous syntactic injection patterns across modules.

Overall, the results demonstrate systematic syntactic inflation at the AST level, significantly enlarging structural footprint and increasing the complexity of syntax-based static analysis.

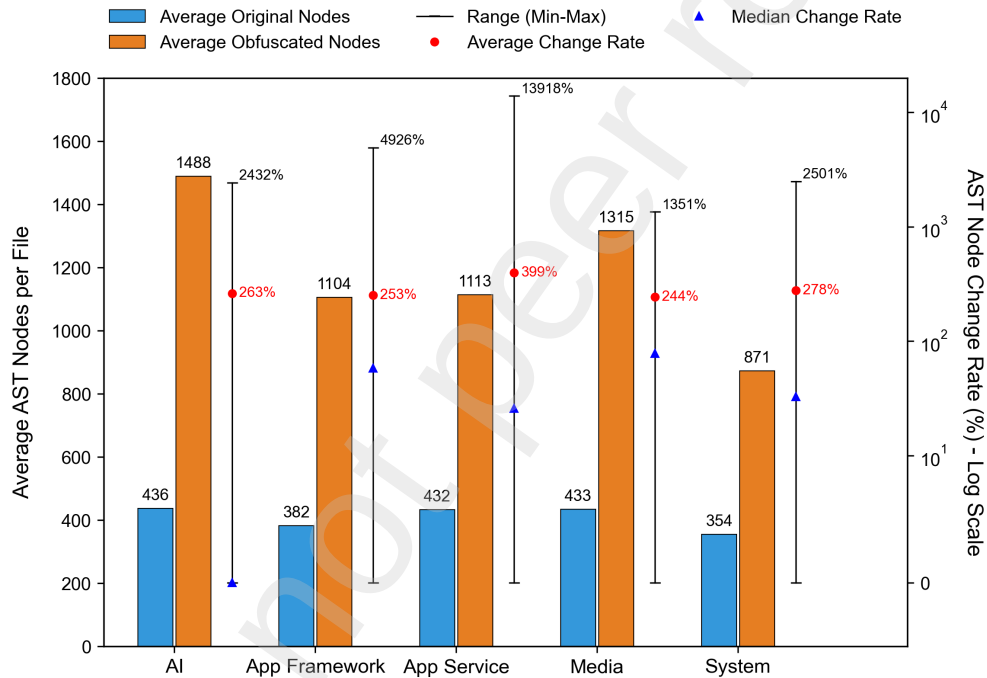


Figure 5: Average AST Node Count Before and After Obfuscation

Code Similarity. Code similarity quantifies the syntactic similarity between the original code and the obfuscated code, serving as a key quantitative metric for evaluating the effectiveness of obfuscation in disrupting structural patterns and implementation details [2]. It measures the extent to which obfuscated code conceals its original structure from static analysis.

In this work, we employ JPLAG, a state-of-the-art open-source framework recently highlighted for its enhanced resilience against automated obfuscation attacks [30]. By utilizing this obfuscation-resilient detector to measure the syntactic similarity between original and obfuscated versions of ArkTS source files, we provide a rigorous assessment of ArkObfuscation’s protection strength. To ensure a fair comparison, identical source files are analyzed before and after obfuscation under consistent conditions.

As shown in Table 6, obfuscation significantly reduces code similarity across all five categories. The average similarity drops to between 25% and 40%, indicating that the syntactic structure has been substantially altered, thereby limiting the ability of even advanced similarity-based detectors to identify the original code.

Table 6: Similarity Statistics Across Different Categories. Metrics are computed for category-wise similarity analysis. Abbreviations: Sim. = Similarity.

Category	Average Sim.	Maximum Sim.	Minimum Sim.	Median Sim.
AI	26.97%	79.44%	7.62%	17.53%
App Framework	36.18%	88.24%	11.18%	35.97%
App Service	37.82%	94.00%	9.91%	31.48%
Media	38.95%	47.83%	26.08%	42.94%
System	39.30%	97.17%	8.80%	36.18%

However, the maximum similarity values remain relatively high (up to 97.17%), suggesting the presence of residual structural patterns that persist despite obfuscation. These include shared framework-level constructs, mandatory lifecycle methods imposed by the ArkTS programming model, and boilerplate code required for platform compatibility. Such elements inherently constrain the degree of syntactic variation and may lead to localized similarity preservation, even under aggressive transformation strategies.

Notably, the median similarity values (ranging from 17.53% to 42.94%) are consistently lower than the averages, indicating that while some samples retain high similarity due to these constrained components, the majority exhibit significant syntactic divergence.

Overall, our results demonstrate that the proposed obfuscation strategy effectively reduces surface-level syntactic similarity while preserving essential functional and framework-level structures. This balance ensures both strong stealth against similarity-based analysis and functional correctness. Consequently, ArkObfusc achieves robust obfuscation performance, exhibiting high resistance to automated code clone detection and reverse engineering techniques.

4.4. ArkTS code’s resistance to reverse analysis after being obfuscated by ArkObfusc(RQ2)

In this section, we extend our evaluation beyond source-code structural metrics to assess the effectiveness of ArkObfusc at two complementary levels: (1) the information-theoretic and semantic dimension, which captures resistance to modern AI-assisted reverse engineering; and (2) the bytecode-level static analysis dimension, which reflects difficulty for traditional static analysis tools targeting compiled artifacts. Together, these dimensions provide a comprehensive picture of how ArkObfusc increases the cost and complexity of reverse engineering under both contemporary and conventional threat models.

4.4.1. Information-Theoretic and Semantic Obfuscation Performance

Beyond structural complexity, effective obfuscation should disrupt the predictability and semantic transparency of code, making it harder for both automated tools and human analysts to infer program intent. We evaluate this aspect using two complementary metrics: entropy (an information-theoretic measure of randomness) and LLM-based semantic evaluation (a proxy for AI-assisted reverse engineering difficulty).

Entropy. Entropy measures the distributional randomness of code elements and is commonly used as an information-theoretic indicator of disorder and unpredictability in program representations. In the context of obfuscation, entropy is often employed to assess whether code transformations increase or decrease the apparent randomness perceived by automated analysis tools.

We compute file-level entropy using an open-source entropy analysis tool on the same representative ArkTS source file used in previous experiments. It is worth noting that TypeScript and ArkTS source code

inherently exhibit relatively low entropy, as they are structured, human-readable programming languages rather than compressed or binary formats. In practice, their entropy values typically range between 4.0 and 5.5.

Our results show that the entropy of the analyzed files slightly decreases after obfuscation. Figure 6 reports the entropy variation before and after obfuscation. This reduction indicates a more concentrated byte and token distribution in the obfuscated code. Such behavior can be attributed to the obfuscation strategy employed by ArkObfuscator, which introduces a large amount of template code, fixed-pattern control-flow constructs, and frequent use of specific keywords such as `if` and `while`. Additionally, the insertion of repetitive jump instructions and standardized control-flow templates further reduces distributional diversity.

Although entropy reduction may appear counterintuitive at first glance, this phenomenon does not imply weakened obfuscation. Instead, it reflects a shift from semantically meaningful variability to structurally repetitive yet analytically complex patterns. The resulting code becomes less predictable at the semantic and control-flow levels, while simultaneously resisting pattern-based deobfuscation and automated reasoning.

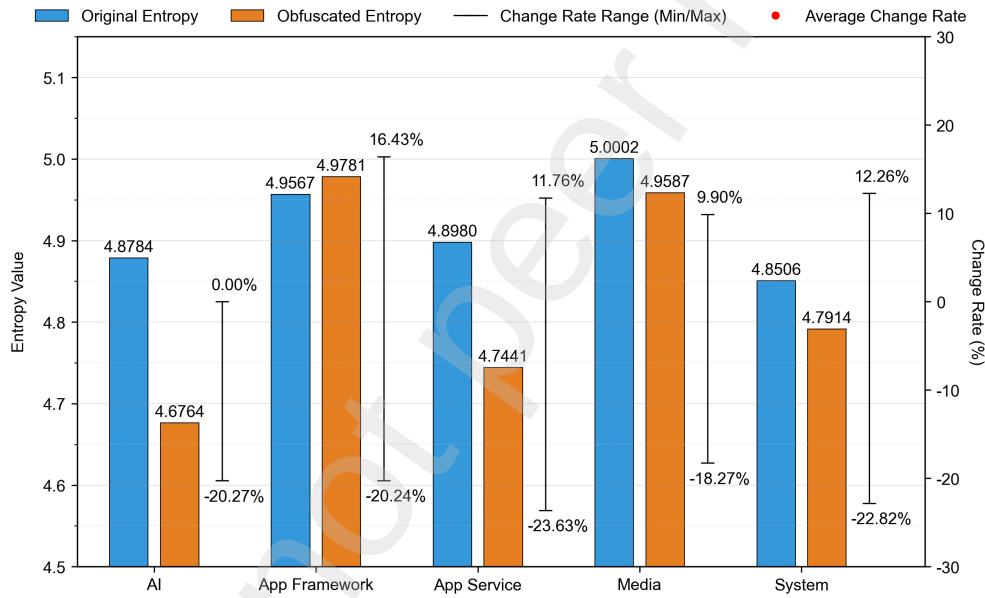


Figure 6: Entropy Metrics Before and After Obfuscation

LLM Metric. To quantitatively evaluate the resilience of obfuscation against AI-assisted reverse engineering, we draw inspiration from Jin et al. [18], who proposed a framework for measuring the resistance of obfuscation techniques using automated analysis tools. Following a similar idea, we employ large language models (LLMs) as semantic-level attackers to assess the extent to which high-level program semantics remain recoverable from source code.

Unlike traditional structural metrics (e.g., cyclomatic complexity or CFG statistics), LLM-based evaluation focuses on the recoverability of program semantics from a comprehension perspective. Code readability and comprehension have long been recognized as key factors affecting program understanding [31]. Recent empirical studies further show that code obfuscation significantly increases the effort required for program comprehension and attack analysis [28].

Specifically, we introduce an adversarial role-playing mechanism in the prompt, requiring the model to simulate the cognitive process of a reverse engineer when analyzing the given code. The scoring criteria

include five evaluation dimensions: Readability, Reverse Engineering Difficulty, Semantic Understanding, Technical Quality, and Total Score.

These dimensions collectively capture whether an automated reasoning agent can correctly infer program intent, reconstruct functional behavior, and maintain coherent technical interpretation. In the context of reverse engineering, stronger semantic recoverability implies weaker obfuscation resistance. Conversely, degradation in LLM comprehension performance indicates improved resilience against AI-assisted analysis.

All evaluations are conducted under a controlled setting using identical prompts, fixed scoring rubrics, and unified metric definitions across three models. This design ensures that score differences are attributable to code transformation rather than prompt variance or evaluation inconsistency.

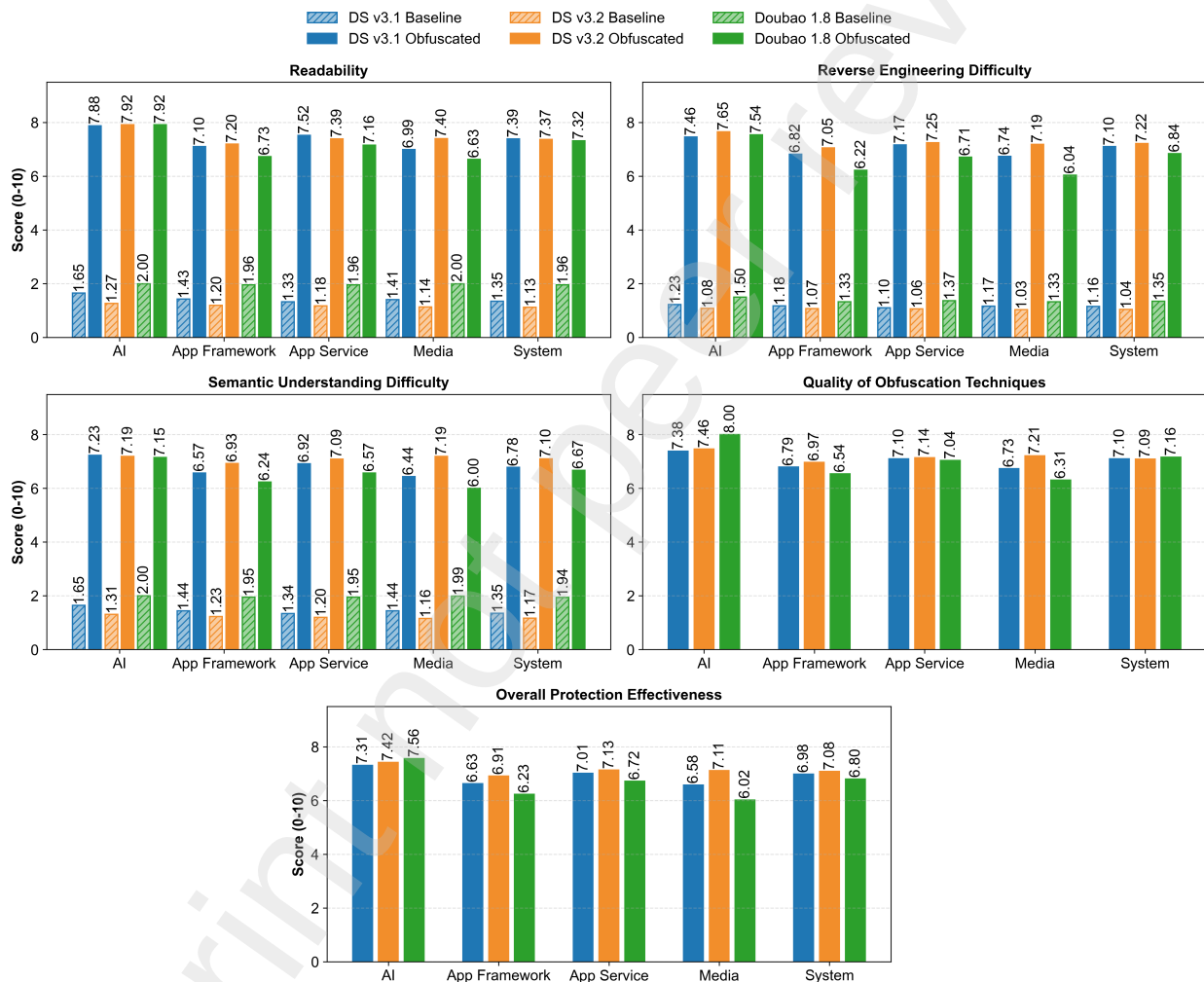


Figure 7: LLM-based Evaluation of Obfuscation Effectiveness

As shown in Figure 7, obfuscation consistently reduces readability and semantic understanding scores across all categories, while substantially increasing perceived reverse-engineering difficulty when compared with the corresponding baseline versions. For technical quality and overall score, which are assessed on the obfuscated versions to evaluate post-transformation coherence and structural integrity, the results remain within a stable and reasonable range. This indicates that the obfuscation process does not introduce excessive syntactic corruption or functional degradation, but primarily affects interpretability rather than

correctness. This separation allows us to distinguish between interpretability degradation and structural validity preservation.

Importantly, the observed trend remains stable across all three evaluated models. Although absolute scores vary slightly due to inherent model characteristics, the directional changes are highly consistent. This cross-model agreement suggests that the degradation effect reflects genuine semantic obfuscation rather than model-specific bias. Moreover, the performance shifts are broadly aligned across heterogeneous project categories, indicating that the resistance effect generalizes beyond particular code domains.

Overall, the LLM-based assessment demonstrates that the obfuscation strategy not only increases structural complexity, but also significantly reduces semantic recoverability under AI-assisted reverse-engineering settings, thereby enhancing practical anti-reverse capability in modern analysis environments.

4.4.2. Byte-level Static Analysis Performance

Since reverse engineering typically targets compiled artifacts rather than source code, we further conduct a bytecode-level analysis on Ark bytecode. Specifically, both the original and obfuscated versions of the program are independently compiled into `.abc` bytecode files, which are then disassembled into textual `.asm` files using the official tool `ark_disasm`, enabling low-level static inspection.

We extract a set of lightweight yet indicative metrics from the disassembled bytecode via pattern matching, focusing on control-flow distortion and structural expansion that directly affect disassembly and static analysis tools:

- **Instruction Count:** It measures overall bytecode expansion introduced by obfuscation.
- **Label Count:** It reflects control-flow fragmentation and the proliferation of jump targets.
- **Jump Count:** It captures the density of explicit control transfers, indicating flattened or redirected control flow.
- **Back-edge Count:** It characterizes loop-like structures and backward jumps that complicate control-flow reconstruction.
- **Function Count:** It reflects function-level structural transformations, such as artificial decomposition or auxiliary method insertion.⁷

As shown in Figure 8, all bytecode-level metrics exhibit substantial increases after obfuscation. In particular, the sharp growth in labels, jump instructions, and back-edges suggests severe control-flow fragmentation and loop distortion, which significantly hinder static analysis and automated reverse engineering. The moderate increase in function count further indicates structural diversification at the method level.

4.5. Performance Comparison(RQ3)

In this section, we evaluate the performance of ArkObfusc by comparing it with existing obfuscation and hardening solutions targeting ArkTS applications.

⁷In §4.3.2, the Cyclomatic Complexity metric analyzes the function count at the source code level, which refers to the number of functions defined in the original source file before compilation; while this section analyzes the function count at the bytecode level, which refers to the number of functions identified in the compiled bytecode after code conversion and compilation.

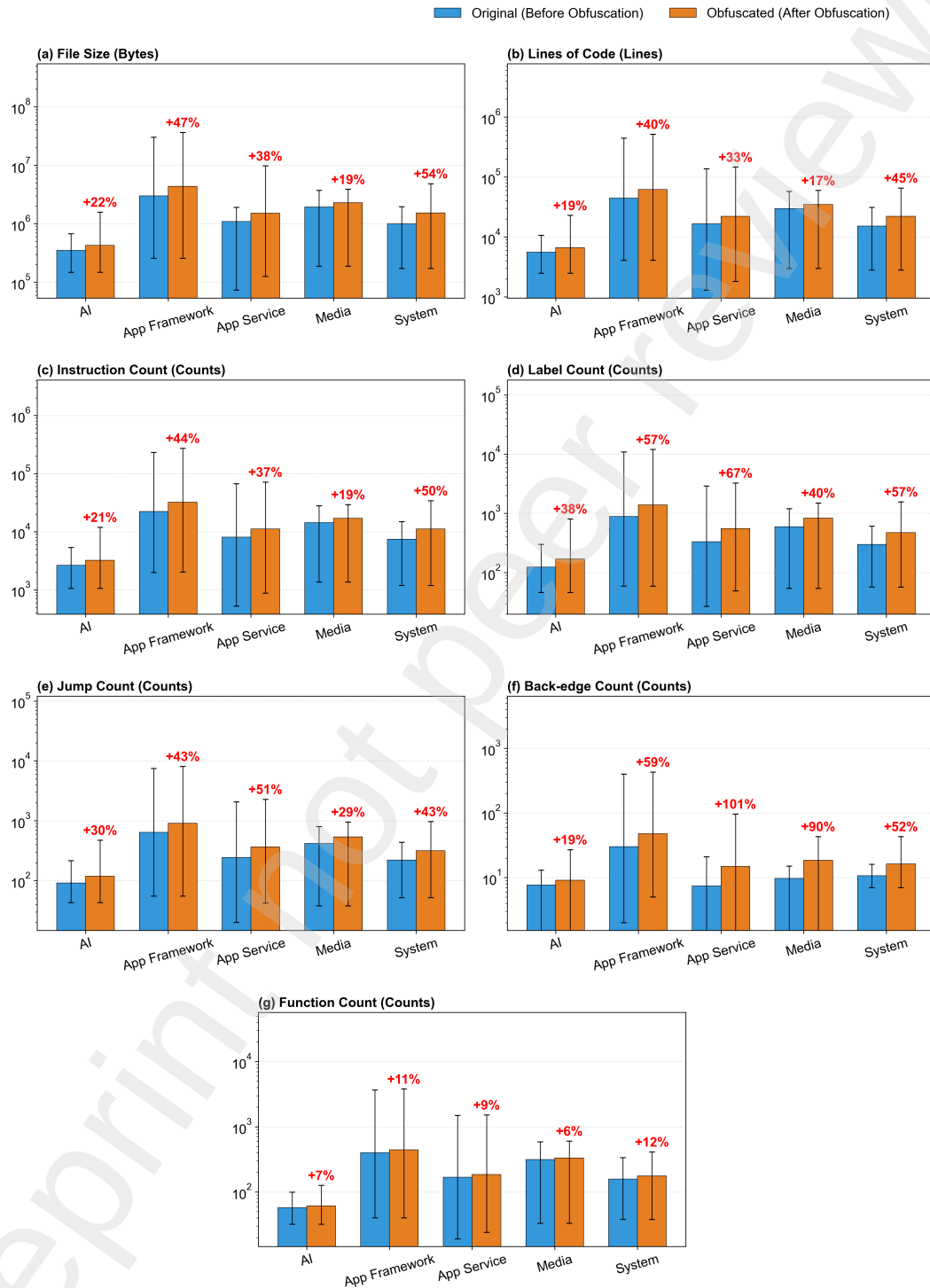


Figure 8: Byte Code Metrics

Baseline Selection. Due to the limited availability of open-source ArkTS obfuscation systems, our quantitative comparison primarily focuses on *ArkGuard*, the official obfuscation tool integrated into the HarmonyOS toolchain (i.e., DevEco Studio)⁸. *ArkGuard* represents the most widely adopted and reproducible baseline in current ArkTS development practice. It mainly supports lightweight obfuscation techniques such as identifier, file name, and function name renaming, while advanced control-flow or semantic-level transformations are not publicly documented.

In addition to *ArkGuard*, several commercial hardening solutions, including *FairGuard*, *NetEase YiDun*, and *BangBang Security*, also claim to provide protection for HarmonyOS applications. However, these systems are closed-source commercial products, and their internal obfuscation strategies, configuration granularity, and transformation scope are not public and uncontrollable. As a result, they cannot be fairly or reproducibly evaluated under a controlled academic setting. Therefore, following common practice in software protection research, we include commercial solutions only in a qualitative capability-level comparison, rather than direct experimental evaluation.

In Figure 9, the gray segments indicate functionalities whose support status cannot be confirmed due to the limited availability of public technical documentation.

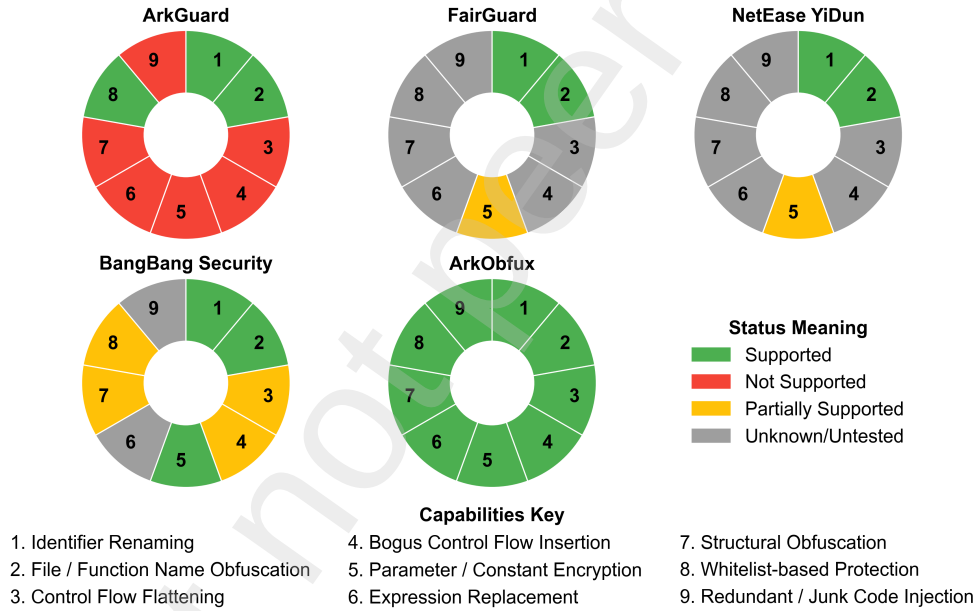


Figure 9: Capability Coverage of ArkObfusc Compared to Existing ArkTS Obfuscation and Hardening Solutions

Methodology. For quantitative evaluation, we apply both *ArkGuard* and *ArkObfusc* to the same benchmark dataset described in Section 4.1. All experiments are conducted in a pairwise manner by comparing each original ArkTS project with its obfuscated counterpart, ensuring consistency across different application categories and function-level roles.

As shown in Figure 10, we compared the official obfuscation tool *ArkGuard* with *ArkObfusc*. We used three metrics:

⁸DevEco Studio is an integrated development environment (IDE) for developing HarmonyOS applications and meta-services. The platform provides AI-assisted programming, compilation and building, real-time UI preview, code debugging, performance tuning, simulators, and other functions.

CFG Complexity Expansion Rate (BC-CER).

$$\text{BC-CER} = \frac{V_{\text{obf}} + E_{\text{obf}}}{V_{\text{orig}} + E_{\text{orig}}}$$

BC-CER measures the structural expansion of the control-flow graph (CFG), where (V) and (E) denote the numbers of basic blocks and edges, respectively. A larger BC-CER indicates that the obfuscated program introduces more control-flow nodes and transitions relative to the original program. When BC-CER (≥ 1), the CFG becomes structurally more complex; the greater the value, the more significant the control-flow inflation. Higher values generally imply increased reverse-engineering cost, as analysts must reason about a denser and more intricate execution structure.

Conditional Jump Inflation Rate (CJIR).

$$\text{CJIR} = \frac{\text{CondJump} * \text{obf}}{\text{CondJump} * \text{orig}}$$

CJIR quantifies the relative increase in conditional branch instructions after obfuscation. A higher CJIR means more predicate-based branching is introduced, which typically reflects techniques such as opaque predicates, control-flow flattening, or branch insertion. When CJIR (≥ 1), additional decision points are injected into the program. Larger values correspond to higher path diversity and greater difficulty for static analysis tools, symbolic execution engines, and human analysts attempting to reconstruct the original logic.

Literal Bytes Expansion Rate (LER).

$$\text{LER} = \frac{\text{LiteralBytes} * \text{obf}}{\text{LiteralBytes} * \text{orig}}$$

LER measures the relative growth of literal data (e.g., string constants and hard-coded values) in the compiled output. When LER (≥ 1), the obfuscated version contains more literal bytes than the original. A higher LER may indicate the introduction of encoded constants, lookup tables, or encrypted payload fragments used to conceal sensitive semantics. Moderate growth can reflect semantic-level protection mechanisms, while excessively large values may imply noticeable binary bloat, potentially affecting storage footprint and runtime performance.

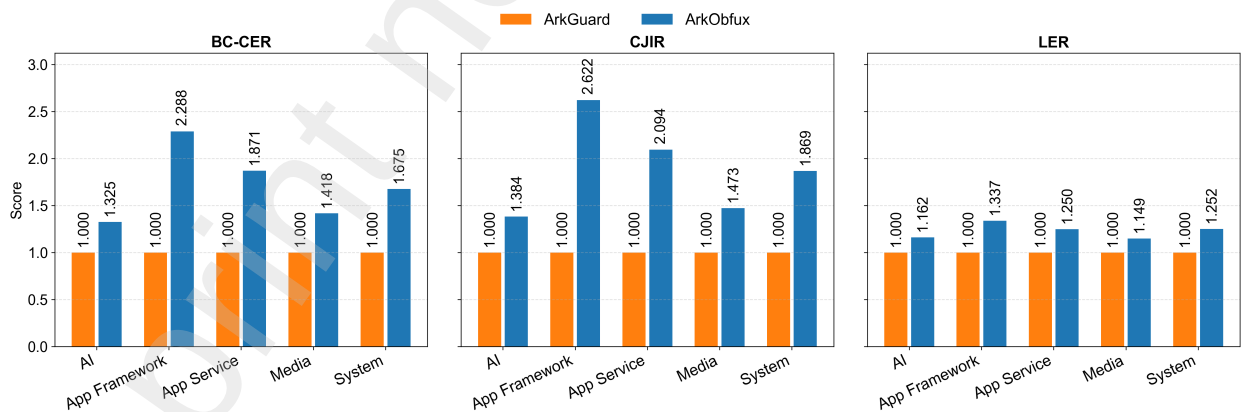


Figure 10: Comparison of obfuscation capabilities between ArkObfusc and ArkGuard. Abbreviations: BC-CER = CFG Complexity Expansion Rate, CJIR = Conditional Jump Inflation Rate, LER = Literal Bytes Expansion Rate.

In our experiments, all three metrics range between 1 and 2, indicating that ArkObfusc introduces measurable structural and semantic expansion while maintaining controlled overhead without excessive code inflation.

5. Related Work

Software reinforcement and code protection techniques have been widely studied to defend applications against reverse engineering and malicious analysis, particularly in mobile and embedded environments. Existing studies can be broadly categorized into code obfuscation techniques, analysis-resistant mechanisms, and system-level protection frameworks. This section reviews representative works in these areas and clarifies the positioning of our multi-layer reinforcement system for HarmonyOS terminal applications.

5.1. Code Obfuscation and Program Transformation

Code obfuscation aims to increase the difficulty of understanding and analyzing software while preserving its original functionality [10]. Prior studies have systematically categorized obfuscation approaches according to different software abstraction levels, including element-level, component-level, and application-level protections [38]. Typical obfuscation techniques include identifier renaming, control-flow restructuring, opaque predicates, and data encoding [10, 13].

More advanced approaches further introduce control-flow flattening, bogus branch insertion, and other semantic-preserving transformations to significantly increase structural complexity [21, 11]. Furthermore, some recent studies have explored inter-procedural obfuscation primitives, such as the fission and fusion mechanisms proposed in KHAOS [40], to disrupt binary diffing by restructuring code across function boundaries.

In mobile application scenarios, obfuscation is often applied at the bytecode or intermediate representation level to accommodate managed runtimes [19]. Several works have demonstrated the effectiveness of control-flow-based obfuscation in increasing resistance to static analysis tools [2]. Empirical studies further investigate the impact of obfuscation on program comprehension and reverse engineering effort [5]. However, most existing techniques are designed for traditional platforms and do not consider the constraints imposed by emerging application frameworks and compilation pipelines.

Recent studies also focus on evaluating and characterizing the effectiveness of obfuscation techniques. Quantitative frameworks have been proposed to measure the quality of obfuscation transformations and analyze their impact on program analysis resistance [18]. In addition, recent surveys investigate detection methods, classification strategies, and broader security implications of code obfuscation in modern software ecosystems [26].

5.2. Resistance to Static and Dynamic Program Analysis

To counter static analysis, prior studies propose increasing control-flow complexity by expanding the size and irregularity of control-flow graphs (CFGs) [10, 21, 20]. Metrics such as node count, edge count, branch density, and cyclomatic complexity are commonly used to quantify obfuscation strength [22, 2]. Foundational research on software protection and obfuscation techniques further systematizes these approaches and their effectiveness in resisting program analysis [8].

Dynamic analysis resistance has been explored through anti-debugging mechanisms, environment fingerprinting, and runtime integrity checks [32]. While these techniques can effectively disrupt dynamic inspection, they often introduce non-negligible performance overhead and remain vulnerable to sophisticated bypass techniques. As a result, recent research trends emphasize balanced designs that jointly consider security effectiveness and runtime efficiency [2].

5.3. System-Level and Multi-layer Protection Frameworks

Beyond individual transformations, several works propose system-level protection frameworks that integrate multiple defensive techniques [8]. These frameworks typically adopt a layered design, combining static obfuscation, runtime checks, and integrity verification to provide comprehensive protection [9].

Such multi-layer approaches exhibit superior robustness compared to single-technique defenses [2]. Recent research [27] further demonstrates that strategic obfuscation layering significantly alters structural code patterns, effectively complicating automated analysis. Empirical studies of real-world mobile applications further indicate that code obfuscation is widely deployed in practice, although many applications rely primarily on lightweight transformations such as identifier renaming or simple control-flow modifications [16, 17]. Nevertheless, existing frameworks primarily target Android or desktop environments, and their design assumptions may not hold for emerging operating systems with distinct application models and execution semantics.

5.4. Protection Techniques for Emerging Operating Systems

The rapid evolution of operating systems and application ecosystems introduces new challenges for software protection [8]. HarmonyOS, with its ArkTS-based development model and distinct compilation toolchain, differs substantially from conventional mobile platforms.

To date, only limited research efforts have investigated program analysis and reinforcement techniques tailored to HarmonyOS terminal applications. Existing studies primarily focus on static analysis tooling for ArkTS, such as the community-developed ArkAnalyzer framework [6], while the impact of control-flow-level reinforcement on program analysis tools within this ecosystem remains underexplored.

5.5. Summary and Our Positioning

In summary, prior work has extensively investigated code obfuscation and analysis-resistant techniques, as well as multi-layer protection frameworks in established platforms. However, there is a lack of system-level reinforcement solutions specifically designed for HarmonyOS terminal applications. Our work addresses this gap by proposing a multi-layer reinforcement system that integrates control-flow reinforcement and runtime protection mechanisms, and by providing a comprehensive evaluation using CFG-level metrics and automated analysis tools.

6. Discussion

This section discusses the overall performance of the proposed system in practical applications, focusing on the effectiveness of the system architecture, design trade-offs and performance impact, the limitations and applicability of the method itself, and further exploring its potential impact and future expansion directions.

6.1. Effectiveness of the System Architecture

From an overall design perspective, the system proposed in this paper does not target a single control flow obfuscation strategy, but rather constructs a multi-dimensional code obfuscation framework that integrates various techniques such as control flow obfuscation, data encryption, structural obfuscation, and control flow flattening. While ensuring the semantic correctness of the ArkTS program, the system can collaboratively modify the program structure at different levels of abstraction, thereby forming a layered protection effect.

At the architectural level, the system models the source code and intermediate representation through a unified program analysis module, providing precise application points for subsequent obfuscation transformations. For example, control flow obfuscation and flattening primarily affect function-level and basic

block-level structures, while data encryption and structural obfuscation more significantly impact variable representation, data access paths, and code organization. This layered and decoupled design allows different obfuscation strategies to be applied independently or combined as needed, avoiding semantic risks caused by mutual interference.

Experimental results show that under the combined effect of multiple obfuscation strategies, the size of the program's control flow graph, branch complexity, and overall structural characteristics all undergo significant changes. In particular, the combination of control flow obfuscation and flattening deconstructs and reorganizes the originally clear execution path, while data encryption and structural obfuscation further weaken the ability of static analysis to recover key semantic information. This demonstrates that the system architecture can effectively support a multi-strategy collaborative obfuscation process, significantly increasing the overall difficulty of reverse engineering the program.

Furthermore, the system adopts a modular design, decoupling program analysis, obfuscation strategy implementation, and metric evaluation. This allows for the addition of new obfuscation operators or adjustments to the combination method without requiring large-scale modifications to the overall framework. This architectural design not only enhances the system's scalability but also provides a solid engineering foundation for subsequent in-depth optimization of the ArkTS language features.

6.2. Design Trade-offs and Performance Considerations

While introducing various techniques such as control flow obfuscation, control flow flattening, data encryption, and structural obfuscation, system design inevitably faces a trade-off between enhanced security and performance overhead. Different obfuscation strategies have different emphases on the program's impact: control flow obfuscation and flattening primarily increase the complexity of the execution path, while data encryption and structural obfuscation introduce additional computational and access costs to some extent.

In terms of design trade-offs, this paper's system does not pursue extreme obfuscation strength in a single dimension, but rather emphasizes the synergy and balance of multiple strategies. For example, data encryption mainly targets sensitive constants and critical data, rather than comprehensively encrypting all data, to avoid unnecessary runtime decryption overhead; structural obfuscation focuses on reorganizing code structure and call relationships, rather than simply increasing code size. This choice allows the system to improve overall protection capabilities while keeping runtime performance impact within a relatively acceptable range.

Experiments and practical observations show that the combined effect of multiple obfuscation strategies significantly enhances the interference effect on static analysis tools, but the program still maintains stable operation under normal execution scenarios. This indicates that the system adopts a relatively pragmatic compromise between security and performance, which is more in line with the realistic performance and energy consumption requirements of mobile and IoT devices. It should be noted that the automated obfuscation process itself increases the time cost of the build phase, especially when multiple strategies are combined. In large ArkTS projects or high-frequency iteration development processes, this factor still needs to be evaluated and weighed in conjunction with actual engineering requirements.

6.3. System Limitations and Applicability Scope

While this system improves overall code protection by combining control flow obfuscation, data encryption, structural obfuscation, and flattening techniques, certain limitations remain. First, while layering multiple strategies significantly increases the difficulty of reverse engineering, it also complicates system configuration and optimization. The impact of different obfuscation strength combinations on performance and maintainability requires repeated experimental verification.

Second, current evaluations primarily rely on static structural metrics (such as CFG size and branch characteristics) to reflect obfuscation effectiveness. These metrics effectively characterize changes at the control flow and structural levels, but lack direct quantitative measures of the protective benefits of data encryption and structural obfuscation in real-world attack scenarios. Therefore, there remains a degree of indirectness between metric results and actual reverse engineering costs, prior empirical research [28] supports that such increases in complexity metrics directly correlate with the extended time and cognitive effort required for human attackers to compromise code.

Regarding applicability, this system is more suitable for core modules or critical business logic in ArkTS applications with high security requirements. These scenarios typically prioritize code protection effectiveness and can tolerate a certain level of build and runtime overhead. For scenarios highly sensitive to performance, energy consumption, or code size, careful selection of obfuscation strategy combinations is necessary, or only enabling some lightweight techniques is required.

6.4. Implications and Future Extensions

Based on the experimental evaluation results presented in this paper, ArkObfux's overall impact can be comprehensively understood from multiple dimensions, including semantic correctness, source code-level obfuscation effectiveness, reverse engineering difficulty, and relative performance compared to existing technologies. This provides a clear direction for future work.

First, the experimental results show that after introducing various transformations such as control flow obfuscation, data encryption, structural obfuscation, and flattening, the ArkTS program can still maintain semantic correctness and functional integrity. This phenomenon indicates that ArkObfux has achieved a good balance between code protection strength and executability in its system design, providing a reliable foundation for its deployment in practical HarmonyOS applications.

Second, from the source code level, the multi-dimensional obfuscation strategy significantly alters the structural characteristics and readability of the ArkTS program. The reorganization of the control flow structure, the disorganization of the code organization, and the encryption of key data make the obfuscated code present a significantly different form from the original implementation in both syntax and structure, effectively increasing the difficulty of human understanding and static analysis.

Furthermore, analysis of reverse engineering scenarios shows that the obfuscated program has significant improvements in both control flow scale and structural complexity. The synergistic effect of control flow obfuscation and flattening disrupts the original clear execution logic, leading to higher time costs and cognitive burdens for reverse engineering tools during path recovery and logic reconstruction.

Finally, comparisons with existing obfuscation techniques show that ArkObfux demonstrates advantages in several obfuscation effectiveness metrics. This advantage does not stem from a single aggressive obfuscation strategy, but rather from the reasonable combination and collaborative design of multiple obfuscation methods, reflecting the effectiveness of customized solutions for the ArkTS and HarmonyOS ecosystems.

Based on the aforementioned analysis, future research can be extended in several key dimensions. First, more sophisticated and diverse functional and stability testing scenarios should be introduced to rigorously validate the system's robustness in production-grade environments. Second, evaluation methodologies should evolve to more closely simulate real-world threat models. Specifically, we intend to supplement our current syntax-based analysis with structural graph-similarity metrics [14] and controlled human-subject experiments [28] to more precisely quantify the resistance of ArkObfux against both automated deobfuscation and manual reverse engineering. Finally, further investigation is required to verify the system's scalability across larger-scale ArkTS projects and diverse application types.

In conclusion, ArkObfux provides a robust, multi-dimensional obfuscation solution for the HarmonyOS ecosystem. Its architectural design and evaluation framework offer a foundational path for advancing code

protection in distributed terminal environments.

7. Conclusion

In this paper, we presented ArkObfuscation, a source-code-level multi-layer reinforcement system for HarmonyOS terminal applications. ArkObfuscation integrates SM2-based sensitive data encryption, bogus control flow insertion, control flow flattening and layout confusion to enhance resistance against both static analysis and dynamic tampering in ArkTS applications.

ArkObfuscation operates directly on ArkTS source files without requiring modifications to the HarmonyOS toolchain. A context-aware policy generator selectively enables transformations according to file roles (e.g., UI entry components, constants, and exported indices), improving compatibility while preserving declarative UI semantics. Extensive evaluation on real-world HarmonyOS projects demonstrates that ArkObfuscation significantly amplifies structural complexity (e.g., CFG size and cyclomatic complexity) and reduces surface-level similarity, while maintaining high compilation success rates and acceptable runtime overhead.

ArkObfuscation provides a practical and scalable protection framework for HarmonyOS developers, enabling systematic integration of multiple obfuscation and encryption techniques in an ArkTS-aware manner. Future work includes further reducing runtime overhead, expanding support for HarmonyOS-specific constructs and libraries, and developing stronger evaluation methodologies that better approximate realistic adversaries, including AI-assisted reverse engineering across broader code contexts.

References

- [1] Hasan Ahmed, Muhammad F. Hyder, Muhammad F. ul Haque, and Paulo C. Santos. Exploring compiler optimization space for control flow obfuscation. *Computers & Security*, 139:103704, 2024. <https://doi.org/10.1016/j.cose.2024.103704>.
- [2] Bjorn Anckaert, Matias Madou, Bruno De Sutter, and Koen De Bosschere. Program obfuscation: A quantitative approach. In *Proc. ACM Workshop on Digital Rights Management (DRM)*, 2007. <https://dl.acm.org/doi/abs/10.1145/1314257.1314263>.
- [3] Gautham Bandrupalli. Ai-driven code refactoring: Using graph neural networks to enhance software maintainability. *arXiv preprint arXiv:2504.10412*, 2025. <https://arxiv.org/abs/2504.10412>.
- [4] Ira D. Baxter, Andrey Yahin, Leonardo Moura, Marcelo Sant'Anna, and Lorraine Bier. Dms: Program transformations for practical scalable software evolution. In *Proc. International Conference on Software Engineering (ICSE)*, 2004. <https://ieeexplore.ieee.org/document/1317484>.
- [5] Mariano Ceccato, Massimiliano Di Penta, Jasvir Nagra, Paolo Falcari, Filippo Ricca, Marco Torchiano, and Paolo Tonella. The effectiveness of source code obfuscation: An experimental assessment. In *Proc. IEEE International Conference on Program Comprehension (ICPC)*, pages 178–187, 2009. <https://doi.org/10.1109/ICPC.2009.5090041>.
- [6] Hao Chen, Dong Chen, Yifan Yang, Liang Xu, Lei Gao, Ming Zhou, Cheng Hu, and Lei Li. ArkAnalyzer: The static analysis framework for openharmony. *arXiv preprint arXiv:2501.05798*, 2025. <https://arxiv.org/abs/2501.05798>.
- [7] R. Cohen, R. David, F. Yger, and F. Rossi. Identifying obfuscated code through graph-based semantic analysis of binary code. *Applied Network Science*, 10(1):49, 2025. <https://doi.org/10.1007/s41109-025-00733-8>.
- [8] Christian Collberg and Jasvir Nagra. *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection*. Addison-Wesley, 2009.
- [9] Christian Collberg and Clark Thomborson. Software tamper resistance: Obstructing static analysis of programs. In *Proc. Network and Distributed System Security Symposium (NDSS)*, 2001. <https://dl.acm.org/doi/book/10.5555/900898>.
- [10] Christian Collberg, Clark Thomborson, and Douglas Low. A taxonomy of obfuscating transformations. Technical Report 148, Department of Computer Science, University of Auckland, 1997. <https://researchspace.auckland.ac.nz/server/api/core/bitstreams/990cb7de-2b44-4a24-acee-82b0efa8e689/content>.
- [11] Christian Collberg, Clark Thomborson, and Douglas Low. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proc. 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1998. <https://dl.acm.org/doi/10.1145/268946.268962>.

- [12] James R. Cordy. The txl source transformation language. *Science of Computer Programming*, 61(3):190–210, 2006. <https://doi.org/10.1016/j.scico.2006.04.002>.
- [13] Giovanni De Pasquale, Federico Nakanishi, Danilo Ferla, and Lorenzo Cavallaro. Ropfuscator: Robust obfuscation with rop. In *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, 2021. <https://arxiv.org/abs/2012.09163>.
- [14] Marc Fyrbiak, Sebastian Wallat, Sven Reinhard, Nicolai Bissantz, and Christof Paar. Graph similarity and its applications to hardware security. *IEEE Transactions on Computers*, 69(4):505–519, 2020.
- [15] Maurice H. Halstead. *Elements of Software Science*. Elsevier North-Holland, 1977. <https://dl.acm.org/doi/abs/10.5555/540137>.
- [16] Mu Hammad, Joshua Garcia, and Sam Malek. A large-scale empirical study on the effects of code obfuscations on android apps and anti-malware products. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 421–431, 2018.
- [17] Antonio Iannillo, Mariano Ceccato, and Massimiliano Di Penta. An empirical study of code obfuscation practices in the google play store. *Empirical Software Engineering*, 30(2), 2025.
- [18] H. Jin, J. Lee, S. Yang, K. Kim, and D.H. Lee. A framework to quantify the quality of source code obfuscation. *Applied Sciences*, 14(12):5056, 2024. <https://doi.org/10.3390/app14125056>.
- [19] Eric Lafortune. Proguard: A java class file shrinker, optimizer, obfuscator, and preverifier. <https://www.guardsquare.com/proguard>.
- [20] Christopher Linn and Saumya Debray. Obfuscation of executable code to improve resistance to static disassembly. In *Proc. ACM Conference on Computer and Communications Security (CCS)*, pages 290–299, 2003. <https://doi.org/10.1145/948109.948149>.
- [21] Dóra László and Ádám Kiss. Obfuscating c++ programs via control flow flattening. In *Proc. European Conference on Software Architecture Workshops (ECSA-W)*, 2016. <https://www.inf.u-szeged.hu/~akiss/pub/fulltext/laszlo2009obfuscating.pdf>.
- [22] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976. <https://ieeexplore.ieee.org/document/1702388>.
- [23] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004. <https://ieeexplore.ieee.org/document/1265817>.
- [24] Hugo D. Menéndez and Guillermo Suárez-Tangil. Obfsec: Measuring the security of obfuscations from a testing perspective. *Expert Systems with Applications*, 210, 2022.
- [25] Paul Oman and Jack Hagemester. Metrics for assessing a software system’s maintainability. In *Proceedings of the IEEE Conference on Software Maintenance*, pages 337–344, 1992. <https://ieeexplore.ieee.org/abstract/document/242525>.
- [26] Tomer Raitsis, Yossi Elgazari, Guy E. Toibin, Yotam Lurie, Shlomi Mark, and Or Margalit. Code obfuscation: A comprehensive approach to detection, classification, and ethical challenges. *Algorithms*, 18(2):54, 2025.
- [27] Sebastian Raubitzek, Sebastian Schrittwieser, Eva Wimmer, and Katharina Mallinger. Obfuscation undercover: Unraveling the impact of obfuscation layering on structural code patterns. *Journal of Information Security and Applications*, 85:103850, 2024.
- [28] Luca Regano, Daniele Canavese, Cataldo Basile, and Marco Torchiano. Empirical assessment of the code comprehension effort needed to attack programs protected with obfuscation. *arXiv preprint arXiv:2511.21301*, 2025. <https://arxiv.org/pdf/2511.21301>.
- [29] Jonathan Salwan, Sébastien Bardin, and Marie-Laure Potet. Symbolic deobfuscation: From virtualized code back to the original. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, pages 372–392, 2018.
- [30] Tolga Sağlam, Sebastian Hahner, Lukas Schmid, and Eric Burger. Obfuscation-resilient software plagiarism detection with jplag. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 216–220, 2024.
- [31] Simone Scalabrino, Gabriele Bavota, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. A comprehensive model for code readability. In *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018.
- [32] Monirul Sharif, Andrea Lanzi, Jonathon Giffin, and Wenke Lee. Impeding malware analysis using conditional code obfuscation. In *Proc. Network and Distributed System Security Symposium (NDSS)*, 2008. https://xueshu.baidu.com/usercenter/paper/show?paperid=1fb234bf38220646686382c75e346116&site=xueshu_se.
- [33] P. K. Tiwari. Malware detection using control flow graphs. In *2024 2nd International Conference on Device Intelligence, Computing and Communication Technologies (DICCT)*, pages 216–220, 2024. <https://doi.org/10.1109/DICCT61038.2024.10532908>.
- [34] S. K. Udupa, S. K. Debray, and M. Madou. Deobfuscation: Reverse engineering obfuscated code. In *Proceedings of the 12th Working Conference on Reverse Engineering (WCRE)*, pages 45–54, 2005.

- [35] Jordy Van den Broeck, Bart Coppens, and Bjorn De Sutter. Flexible software protection. *arXiv preprint arXiv:2012.12603*, 2020.
- [36] Eelco Visser. *Program transformation with Stratego/XT: Rules, strategies, tools, and systems in Stratego/XT 0.9*, pages 216–238. Springer, 2004. https://doi.org/10.1007/978-3-540-25935-0_10.
- [37] Xiaohu Xiong, Zhiping Sha, Huazhang Shu, and Ran Wei. Code obfuscation based on deep integration. *The Computer Journal*, 68(12):1980–1995, 2025. <https://academic.oup.com/comjnl/article-abstract/68/12/1980/8229674>.
- [38] Hongyu Xu, Yang Zhou, Jiang Ming, and Michael R. Lyu. Layered obfuscation: A taxonomy of software obfuscation techniques for layered security. *Cybersecurity*, 3(9), 2020. <https://doi.org/10.1186/s42400-020-00049-3>.
- [39] Khaled Yakdan, Sebastian Dechand, Elmar Gerhards-Padilla, and Matthew Smith. No more gotos: Decompilation using pattern-independent control flow structuring. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2015. https://net.cs.uni-bonn.de/fileadmin/ag/martini/Staff/yakdan/dream_ndss2015.pdf.
- [40] Peng Zhang, Kai Zeng, Yong Kang, Chao Wu, Dongpeng Yu, Wei Wang, Miao Peng, Yong Lai, and Zhi Wang. Khaos: The impact of inter-procedural code obfuscation on binary diffing techniques. In *Proceedings of the 2023 International Conference on Code Generation and Optimization (CGO)*, pages 1–15, 2023.